

Acute Myocardial Infarction - 30-day mortality

Data analysis and prognostic model building.

Arhin Claribel Akua Tawiah, Bragagnolo Sofia, Guccione Anna, Maia Tavora Leticia

Contents

Question 1	1
Introduction	2
Data loading	2
Data preparation	4
Exploring the data	6
Visual summary	10
Question 2	19
Crude odds and univariable analysis	19
Evaluating the linearity assumption for continuous variables	20
Question 3	22
Preprocessing	22
Full Models	23
Variable Selection	26
Multicollinearity Check	37
Question 4	38
Internal Validation	39
External Validation	44
Conclusion	48

Question 1

Start with an initial data analysis, comparing patients dead versus patients alive at 30 days. Insert also a column with the total population descriptive statistics. Include also a new variable, the Body Mass Index

Introduction

Acute Myocardial Infarction (MI) (“heart attack”) is caused by the formation of a clot in one of the coronary arteries that supply blood to the heart muscle. Mortality is substantial in the period immediately after the event (and during the years after surviving the initial infarction).

The final aim of this project is to predict the probability of 30-day mortality following an AMI.

Therefore, the objective of this first part of the analysis is to explore the characteristics of the patients included in the study and compare subjects who were alive at 30 days with those who died within 30 days.

Before fitting regression models, it is important to understand how the variables are distributed, whether there are possible differences between the two groups, and whether the data contain unusual values or missing information. This initial exploration also helps us identify variables that could later become important predictors in the prognostic model. Another important step is the creation of the Body Mass Index (BMI), which is not directly available in the dataset but can be calculated using height and weight.

Data loading

```
data <- read_excel('./AMI_data.xlsx')
```

Dimensions

The dataset initially includes 3661 patients and 21 variables (20 candidate predictors + 1 outcome variable).

```
data.frame(
  'patients' = nrow(data),
  'factors' = ncol(data)) %>%
  kable(col.names = c("# of patients", "# of prognostic factors"), caption = "Dataset dimensions") %>%
  kable_styling(full_width = FALSE, bootstrap_options = c("bordered"))
```

Table 1: Dataset dimensions

# of patients	# of prognostic factors
3661	21

Missing values

Checking for missing values:

```
colSums(is.na(data))
```

```
## day30 sex age a65 killip sho dia hyp hrt ant pmi hei wei
##      0   0   0   0   0     0   0   0   0   0   0   0   0
## htn  smk lip pan  fam ste  st4  ttr
##      0   0   0   0   0     0   0   0
```

There is no missing values in the data.

Dataset description

```
head(data)
```

```
## # A tibble: 6 x 21
##   day30   sex   age   a65 killip   sho   dia   hyp   hrt   ant   pmi   hei   wei   htn   smk
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     0     0  70.3     1     1     0     0     0     0     1     0  177.  84     1     3
## 2     0     0  59.8     0     1     0     1     0     0     1     0  172  115     1     1
## 3     0     0  59.0     0     1     0     0     0     1     0     0  170   76     1     1
## 4     1     1  80.4     1     1     0     0     0     1     0     0  155.  50     0     3
## 5     0     0  64.8     0     1     0     0     1     0     0     0  167  97.4     0     1
## 6     0     0  53.9     0     1     0     0     0     0     1     0  171  100     1     1
## # i 6 more variables: lip <dbl>, pan <dbl>, fam <dbl>, ste <dbl>, st4 <dbl>, ttr <dbl>
```

```
str(data)
```

```
## tibble [3,661 x 21] (S3: tbl_df/tbl/data.frame)
## $ day30 : num [1:3661] 0 0 0 1 0 0 0 0 0 0 ...
## $ sex   : num [1:3661] 0 0 0 1 0 0 0 0 0 1 ...
## $ age   : num [1:3661] 70.3 59.8 59 80.4 64.8 ...
## $ a65   : num [1:3661] 1 0 0 1 0 0 0 1 1 0 ...
## $ killip: num [1:3661] 1 1 1 1 1 1 1 2 1 1 ...
## $ sho   : num [1:3661] 0 0 0 0 0 0 0 0 0 0 ...
## $ dia   : num [1:3661] 0 1 0 0 0 0 0 0 0 0 ...
## $ hyp   : num [1:3661] 0 0 0 0 1 0 0 0 0 0 ...
## $ hrt   : num [1:3661] 0 0 1 1 0 0 0 0 0 0 ...
## $ ant   : num [1:3661] 1 1 0 0 0 1 0 1 1 0 ...
## $ pmi   : num [1:3661] 0 0 0 0 0 0 0 0 0 0 ...
## $ hei   : num [1:3661] 177 172 170 155 167 ...
## $ wei   : num [1:3661] 84 115 76 50 97.4 100 82 90.9 96.4 61 ...
## $ htn   : num [1:3661] 1 1 1 0 0 1 0 1 1 1 ...
## $ smk   : num [1:3661] 3 1 1 3 1 1 2 2 2 2 ...
## $ lip   : num [1:3661] 1 0 0 0 0 1 0 0 0 1 ...
## $ pan   : num [1:3661] 0 0 0 0 1 0 0 0 0 0 ...
## $ fam   : num [1:3661] 0 1 1 0 1 1 0 1 1 1 ...
## $ ste   : num [1:3661] 1 6 3 3 2 4 3 7 8 3 ...
## $ st4   : num [1:3661] 0 1 0 0 0 0 0 1 1 0 ...
## $ ttr   : num [1:3661] 1 0 0 0 1 0 1 1 1 1 ...
```

Although the raw dataset encodes every variable numerically, they fundamentally belong to different statistical types:

Variable Name	Clinical Description	Statistical Type	Coding / Units in Raw Data
day30	30-day vital status (Primary Outcome)	Categorical (Binary)	0 = Alive, 1 = Dead
sex	Patient biological sex	Categorical (Binary)	0 = Male, 1 = Female

Variable Name	Clinical Description	Statistical Type	Coding / Units in Raw Data
age	Age at presentation	Numerical (Continuous)	Years
a65	Age threshold indicator	Categorical (Binary)	0 = Under 65, 1 = 65 or older
killip	Killip class (Heart failure severity)	Categorical (Ordinal)	1 = None, 2 = Rales/S3, 3 = Pulmonary edema, 4 = Shock
sho	Cardiogenic shock status	Categorical (Binary)	0 = No, 1 = Yes (Killip class 3 or 4)
dia	Diabetes mellitus history	Categorical (Binary)	0 = No, 1 = Yes
hyp	Hypotension status	Categorical (Binary)	0 = No, 1 = Yes (Systolic Blood Pressure < 100 mmHg)
hrt	Tachycardia status	Categorical (Binary)	0 = No, 1 = Yes (Pulse rate > 80 bpm)
ant	Infarction location	Categorical (Binary)	0 = Other, 1 = Anterior infarct
pmi	Previous myocardial infarction	Categorical (Binary)	0 = No, 1 = Yes
hei	Height	Numerical (Continuous)	Centimeters (cm)
wei	Weight	Numerical (Continuous)	Kilograms (kg)
htn	Hypertension history	Categorical (Binary)	0 = No, 1 = Yes
smk	Smoking status	Categorical (Nominal)	1 = Never, 2 = Ex-smoker, 3 = Current smoker
lip	Hypercholesterolaemia	Categorical (Binary)	0 = No, 1 = Yes
pan	Previous angina	Categorical (Binary)	0 = No, 1 = Yes
fam	Family history of MI	Categorical (Binary)	0 = No, 1 = Yes
ste	Number of ECG leads with ST elevation	Numerical (Count)	Number of leads
st4	Extensive ST elevation threshold	Categorical (Binary)	0 = 4 leads or fewer, 1 = More than 4 leads
ttr	Time to pain relief	Categorical (Binary)	0 = 1 hour or less, 1 = Greater than 1 hour

Consequently, treating these variables purely as numerical would invalidate regression assumptions. To ensure a clear interpretation these should be converted into factors.

Data preparation

BMI variable

We create the BMI variable following the formula: $BMI = \frac{weight(kg)}{height(m)^2}$

```
data$bmi <- data$wei/(data$hei/100)^2
```

Factors labels

```
pretty_labels <- list(
  sex ~ "Sex",
  age ~ "Age (years)",
  bmi ~ "Body Mass Index (kg/m^2)",
  killip ~ "Killip class",
  sho ~ "Shock (Killip 3/4)",
  dia ~ "Diabetes",
  hyp ~ "Hypotension (SBP < 100)",
  hrt ~ "Tachycardia (pulse > 80)",
  ant ~ "Anterior infarct location",
  pmi ~ "Previous MI",
  htn ~ "Hypertension history",
  smk ~ "Smoking status",
  lip ~ "Hypercholesterolaemia",
  pan ~ "Previous angina",
  fam ~ "Family history of MI",
  ste ~ "ST elevation (n leads)",
  st4 ~ "ST elevation > 4 leads",
  ttr ~ "Time to pain relief > 1h"
)
status_colors <- c("Alive" = "#519e8a", "Dead" = "#E5625E")
```

Converting to factors

We convert variables into factors for easier interpretation:

```
ami_data <- data %>%
  mutate(
    day30 = factor(day30, levels = c(0, 1), labels = c("Alive", "Dead")),
    sex = factor(sex, levels = c(0, 1), labels = c("Male", "Female")),

    across(
      c(a65, sho, dia, hyp, hrt, ant, pmi, htn, lip, pan, fam, st4, ttr),
      ~ factor(.x, levels = c(0, 1), labels = c("No", "Yes"))
    ),

    killip = factor(killip, levels = c(1, 2, 3, 4)),
    smk = factor(smk, levels = c(1, 2, 3), labels = c("Never", "Ex", "Current"))
  )
head(ami_data)
```

```
## # A tibble: 6 x 22
##   day30 sex    age a65  killip sho  dia  hyp  hrt  ant  pmi  hei  wei htn  smk
##   <fct> <fct> <dbl> <fct> <fct> <fct> <fct> <fct> <fct> <fct> <fct> <dbl> <dbl> <fct> <fct>
## 1 Alive Male   70.3 Yes    1     No   No   No   No   Yes  No   177.  84   Yes  Current
```

```
## 2 Alive Male      59.8 No    1      No    Yes    No    No    Yes    No    172 115    Yes    Never
## 3 Alive Male      59.0 No    1      No    No     No    Yes    No    No    170  76    Yes    Never
## 4 Dead  Female    80.4 Yes   1      No    No     No    Yes    No    No    155.  50    No     Current
## 5 Alive Male      64.8 No    1      No    No     Yes   No     No    No    167  97.4 No     Never
## 6 Alive Male      53.9 No    1      No    No     No    No     Yes   No    171  100   Yes    Never
## # i 7 more variables: lip <fct>, pan <fct>, fam <fct>, ste <dbl>, st4 <fct>, ttr <fct>,
## #   bmi <dbl>
```

Note that to avoid structural multi-collinearity, `a65`, `sho`, `hei`, `wei` and `st4` are excluded from the final multivariable model.

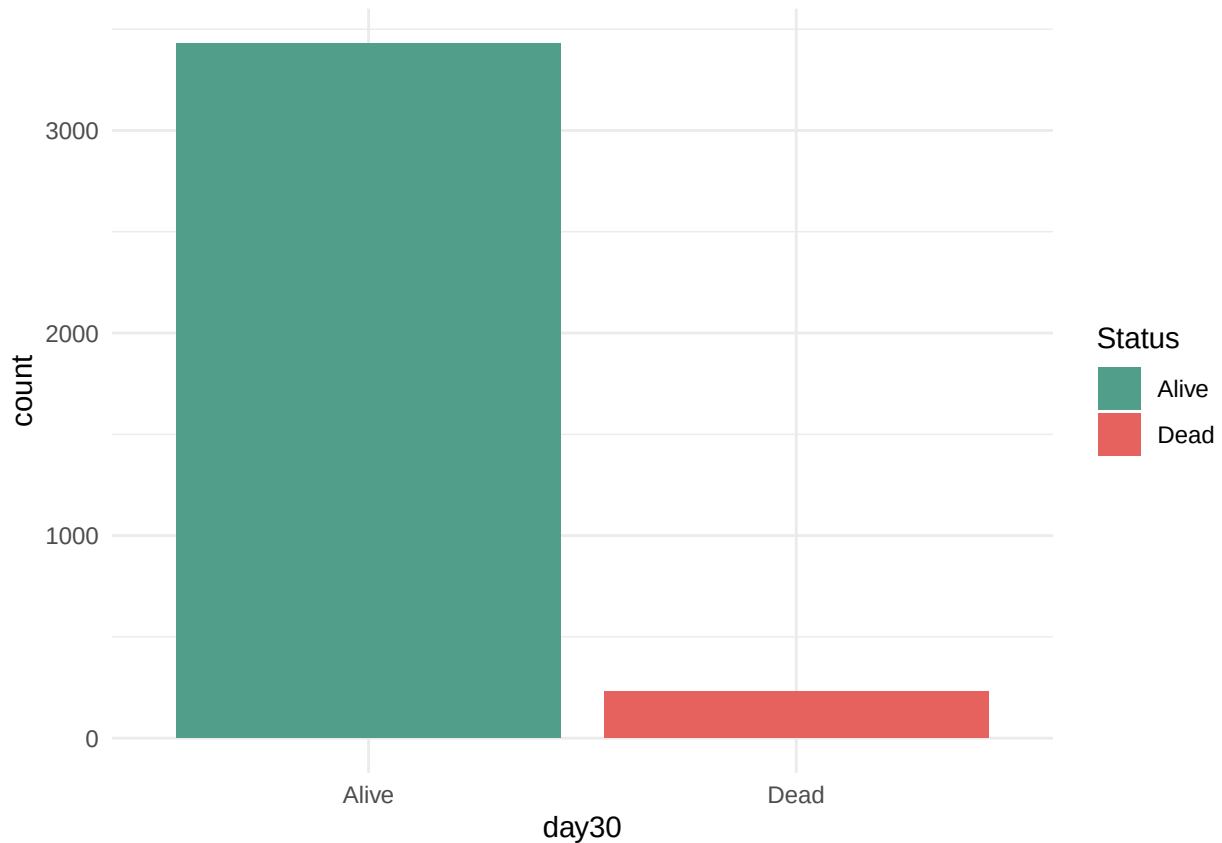
- `a65` is a binary age-derived indicator (age ≥ 65) and is therefore excluded to avoid redundancy with continuous variable `age`
- `hei` and `wei` are used only to compute BMI and are not treated as separate candidate predictors in the predictive model
- `sho` is derived from Killip (Shock = Killip 3 or 4)
- `st4` is derived from `ste`

Note that we still included `sho` and `st4` in the exploratory steps.

Exploring the data

From this initial distribution for a 30-day window, we can see a substantial class imbalance. Surviving patients significantly outnumber deceased patients.

```
ggplot(ami_data) +
  geom_bar(aes(x = day30, fill = day30), stat = "count") +
  scale_fill_manual(name = "Status", values = status_colors) +
  theme_minimal()
```



Cumulative incidence

This analysis uses `binom.exact` from the `epitools` package, which calculates the exact binomial confidence interval for the mortality rate.

```
n_dead <- sum(ami_data$day30 == "Dead")
n_total <- nrow(ami_data)
inc <- binom.exact(n_dead, n_total)
data.frame(
  'Deaths' = n_dead,
  'Total' = n_total,
  'Perc' = round(inc$proportion * 100, 2),
  'CI lower' = round(inc$lower * 100, 2),
  'CI upper' = round(inc$upper * 100, 2)
) %>% kable(col.names = c("Deaths", "Total cohort (N)", "Incidence (%)", "95% CI lower", "95% CI upper"),
  kable_styling(full_width = FALSE, bootstrap_options = c("bordered"))
```

Table 3: Cumulative 30-Day Mortality Incidence

Deaths	Total cohort (N)	Incidence (%)	95% CI lower	95% CI upper
231	3661	6.31	5.54	7.15

The 30-day mortality rate after acute myocardial infarction is relatively low (6.31%), indicating that most

patients survived the acute phase. However, this mortality rate remains clinically important because 1 in approximately 16 patients died within 30 days ($100/6.31=15.848 \approx 16$)

Outcome balance and events per variable (EPV)

To safely build our prognostic regression model, we must assess the **Events per Variable (EPV)**. For a reliable multivariable logistic regression model that avoids severe overfitting, guidelines generally recommend a minimum baseline of 10 to 15 events (deaths) per estimated parameter.

After excluding redundant and derived variables (`a65`, `sho`, `st4`, `hei`, and `wei`), our final model consists of **16 unique candidate predictors**. With 231 total deaths, our dataset yields an EPV of approximately **14.44** per variable.

```
n_predictors <- 16
epv <- n_dead / n_predictors
data.frame(
  'Deaths' = n_dead,
  'Predictors' = n_predictors,
  'EPV' = round(epv, 2),
  'Recommended minimum' = '10-15'
) %>% kable(caption = 'EPV assessment for candidate prognostic predictors') %>%
  kable_styling(full_width = FALSE, bootstrap_options = c('bordered'))
```

Table 4: EPV assessment for candidate prognostic predictors

Deaths	Predictors	EPV	Recommended.minimum
231	16	14.44	10-15

Note on outcome balance: $\approx 6.31\%$ of the sample are deaths. If the death proportion is below 10–15%, rare predictors may lead to separation issues in logistic regression.

Descriptive statistics

To compare baseline characteristics between patients who survived and those who died within 30 days, we utilize the Standardized Mean Difference (SMD) rather than traditional p-values.

In large clinical datasets, traditional p-values are highly sensitive to sample size and can flag trivial differences as “highly statistically significant.”

Interpeting SMD thresholds:

- negligible difference < 0.1
- small difference 0.1-0.3
- moderate difference 0.3-0.5
- large (clinically substantial) difference > 0.5

Here we present the table with the characteristics ordered by the absolute value of the SMD.

```
tbl_summary(ami_data, by=day30,
            include = c(age, killip, sho, ant, hrt, ste, st4, smk, hyp, pmi, sex, bmi, pan, ttr, dia, f,
            label = pretty_labels) %>% add_overall() %>% add_stat_label() %>% bold_labels() %>% add_diff
            modify_spanning_header(all_stat_cols() ~ "***After 30 days**") %>% modify_caption(
            "**Table 1. Baseline characteristics by 30-day vital status**"
            )
```

1. Large differences (SMD > 0.5)

- Age (−0.9): deceased patients are older (median of 73 years) compared to those who survived (median of 60 years). The negative SMD for age means the “Alive” group has a lower value than the “Dead” group (The Dead group is much older). This indicates a substantial difference in age distribution between patients who survived and those who died within 30 days.
- Killip (0.67): severe heart failure at presentation is a massive indicator of mortality

2. Moderate differences (SMD 0.3-0.5)

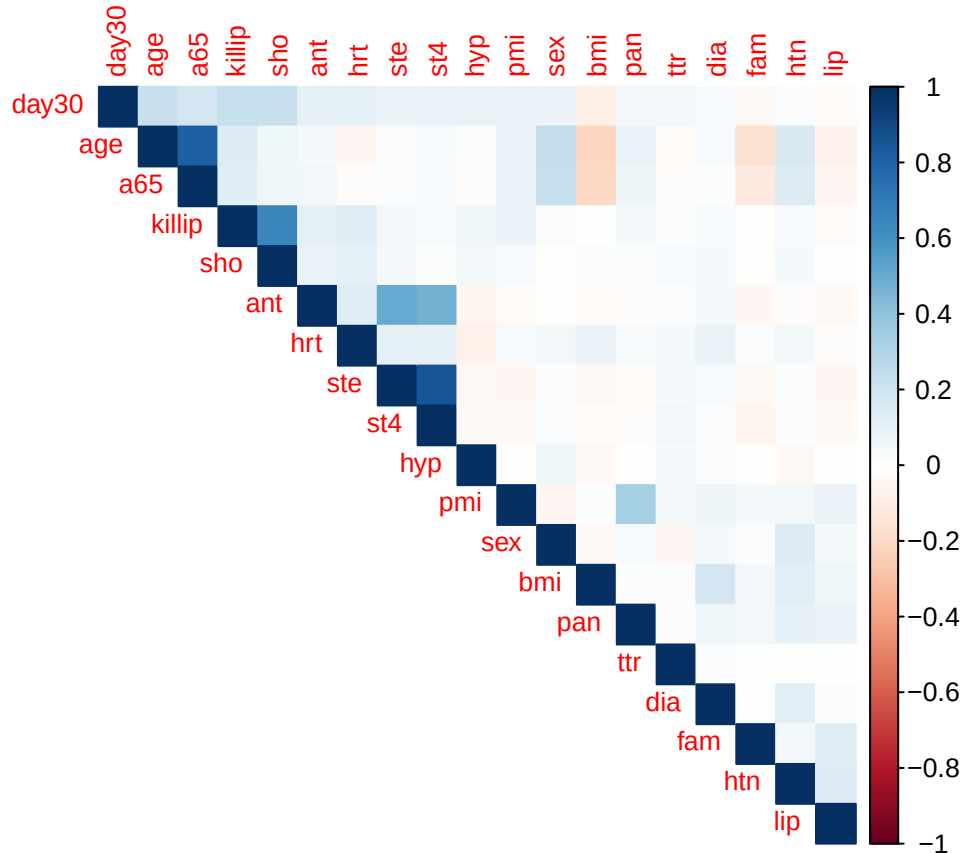
- Deceased patients presented much higher rates of **sho** (13% vs. 0.9%), **ant** (57% vs. 35%), **hrt** (51% vs. 30%), and **hyp** (19% vs. 8.0%).
- BMI (0.32): shows a moderate positive difference. Surviving patients had a higher median BMI (26.9 kg/m²) than those who died (25.3 kg/m²)

Correlation matrix

We convert our factored variables into numeric values. We include the variables targeted for exclusion (sho, st4, a65) to demonstrate their underlying collinearity.

The correlation matrix clearly exposes the redundancy of the derived clinical variables, validating the exclusion: * age and a65 show a high correlation (a65 is a binary split of the continuous age variable) * killip and sho show a high correlation (confirming that sho is formed by Killip classes 3 and 4) * ste and st4 show a high correlation (st4 is redundant).

```
corr_data <- ami_data %>%
  select(day30, age, a65, killip, sho, ant, hrt, ste, st4, hyp, pmi, sex, bmi, pan, ttr, dia, fam, htn,
  mutate(across(everything(), ~ as.numeric(.x)))
corr_matrix <- cor(corr_data, use = "complete.obs")
corrplot(corr_matrix, method = "color", type = "upper", tl.cex = 0.8)
```



```
print(round(corr_matrix[c("age", "a65", "killip", "sho", "ste", "st4"),
                        c("age", "a65", "killip", "sho", "ste", "st4")], 2))
```

```
##      age  a65 killip  sho  ste  st4
## age   1.00 0.82   0.14 0.07 0.01 0.03
## a65   0.82 1.00   0.13 0.07 0.03 0.04
## killip 0.14 0.13   1.00 0.65 0.06 0.03
## sho    0.07 0.07   0.65 1.00 0.04 0.02
## ste    0.01 0.03   0.06 0.04 1.00 0.85
## st4    0.03 0.04   0.03 0.02 0.85 1.00
```

Visual summary

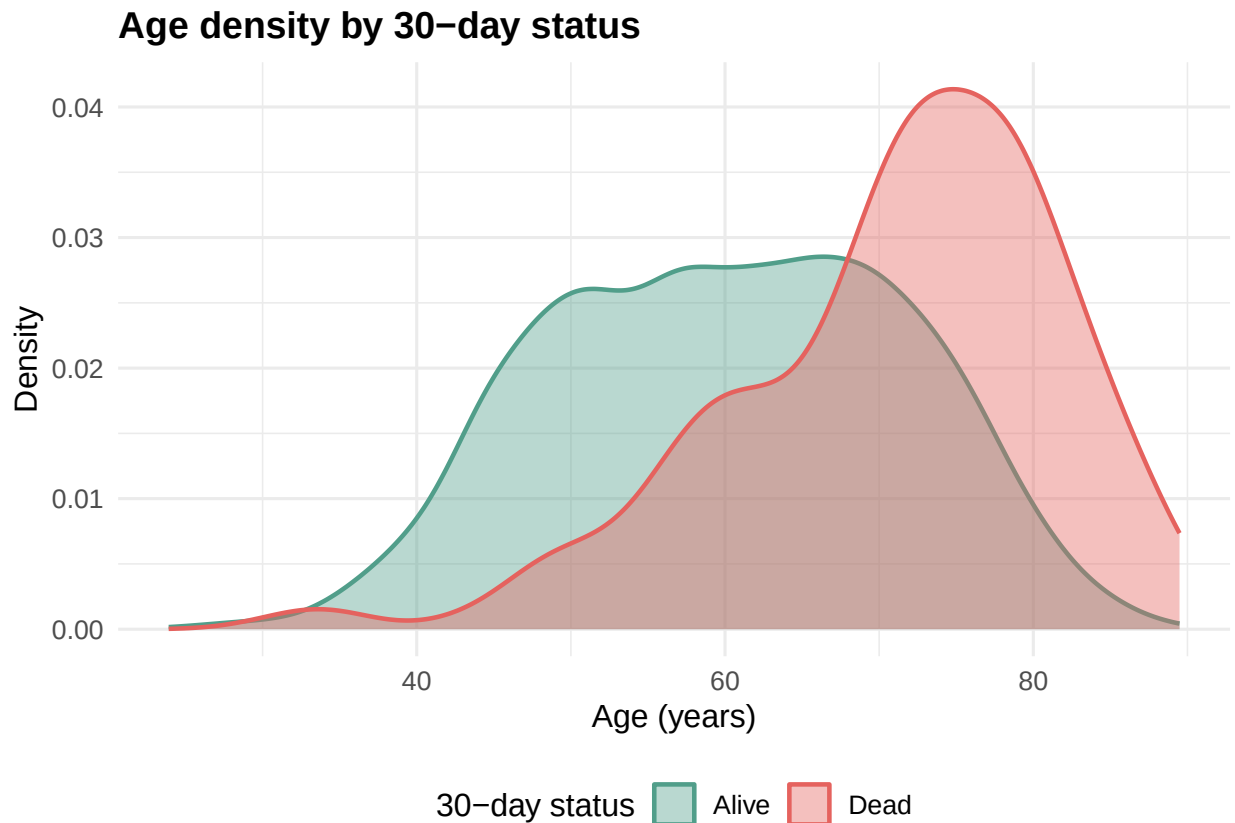
This section presents graphical summaries of key predictors and their relationship with 30-day mortality, helping to identify distributional patterns and group differences before model building.

Age

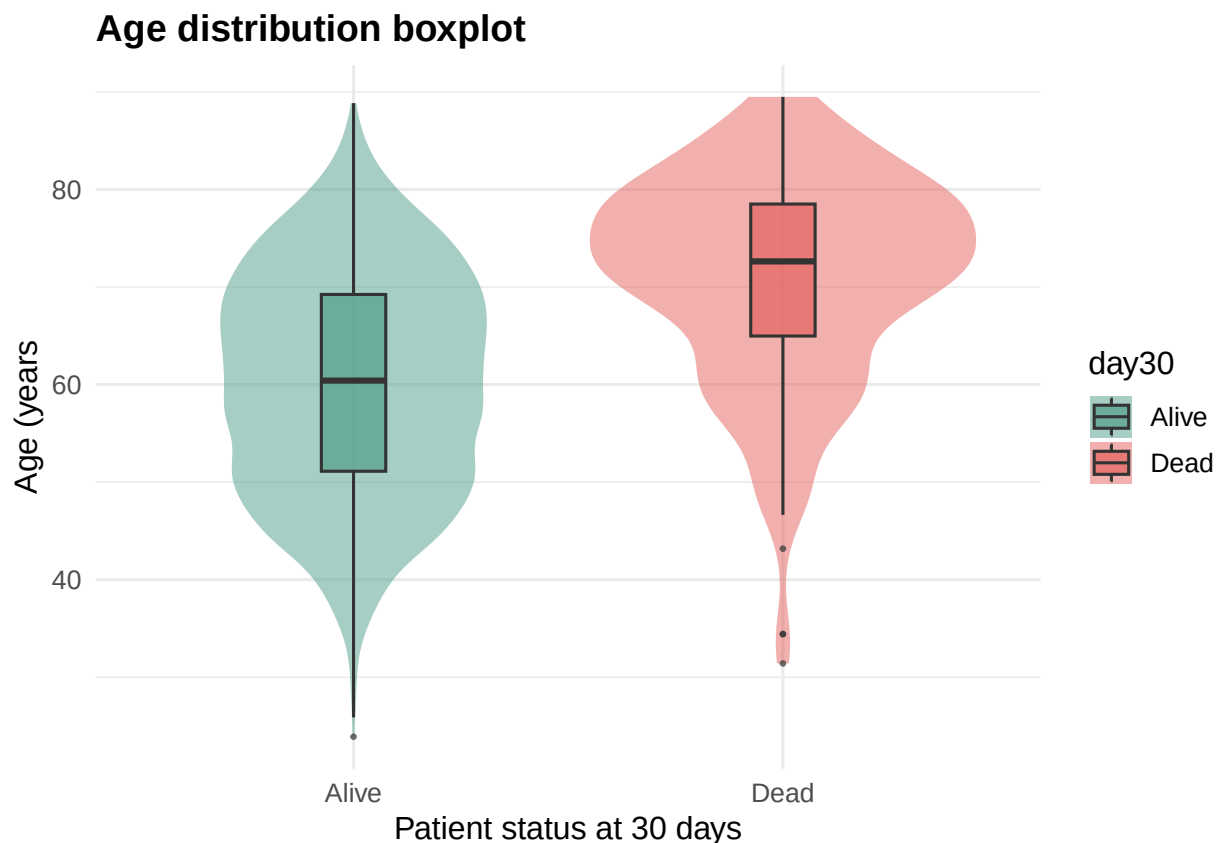
From the plots below we can see how older age is strongly associated with 30-day mortality. Patients who died were substantially older, suggesting age is a major prognostic factor after MI.

```
ggplot(ami_data, aes(x = age, fill = day30, colour = day30)) +
  geom_density(alpha = 0.4, linewidth = 0.8) +
```

```
scale_fill_manual(values = status_colors) +
scale_colour_manual(values = status_colors) +
labs(title = "Age density by 30-day status", x = "Age (years)", y = "Density",
      fill = "30-day status", colour = "30-day status") +
theme_minimal(base_size = 12) +
theme(legend.position = "bottom", plot.title = element_text(face = "bold"))
```



```
ggplot(ami_data, aes(x = day30, y = age, fill = day30)) +
  geom_violin(alpha = 0.5, trim = TRUE, colour = NA) +
  geom_boxplot(width = 0.15, outlier.size = 0.5, alpha = 0.7) +
  scale_fill_manual(values = status_colors) +
  labs(title = "Age distribution boxplot", x = "Patient status at 30 days", y = "Age (years)") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "right", plot.title = element_text(face = "bold"))
```



Killip

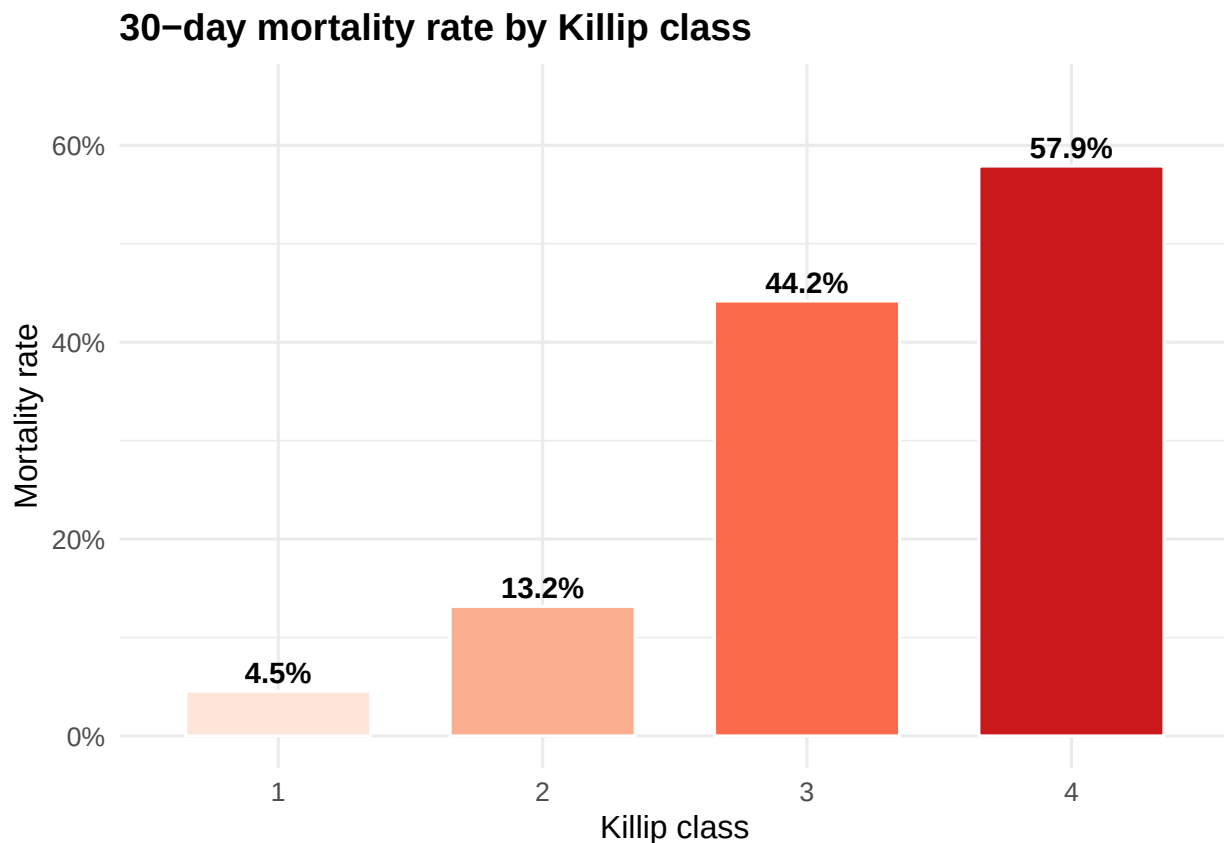
Mortality increases with increasing Killip class, higher Killip class (3,4) reflects more severe heart failure after MI.

We can see that Killip IV has only 19 patients, of which 11 are deaths. This suggests potential instability in the estimated odds ratio for Killip IV due to the small sample size.

```
killip_tab <- ami_data %>%
  group_by(killip) %>%
  summarise(
    Alive = sum(day30 == "Alive"),
    Dead = sum(day30 == "Dead"),
    Total = n(),
    Mortality_rate = round(100 * Dead / Total, 1)
  )
killip_tab
```

```
## # A tibble: 4 x 5
##   killip Alive  Dead Total Mortality_rate
##   <fct> <int> <int> <int>      <dbl>
## 1 1      3022   144   3166         4.5
## 2 2       376    57    433        13.2
## 3 3        24    19     43        44.2
## 4 4         8    11     19        57.9
```

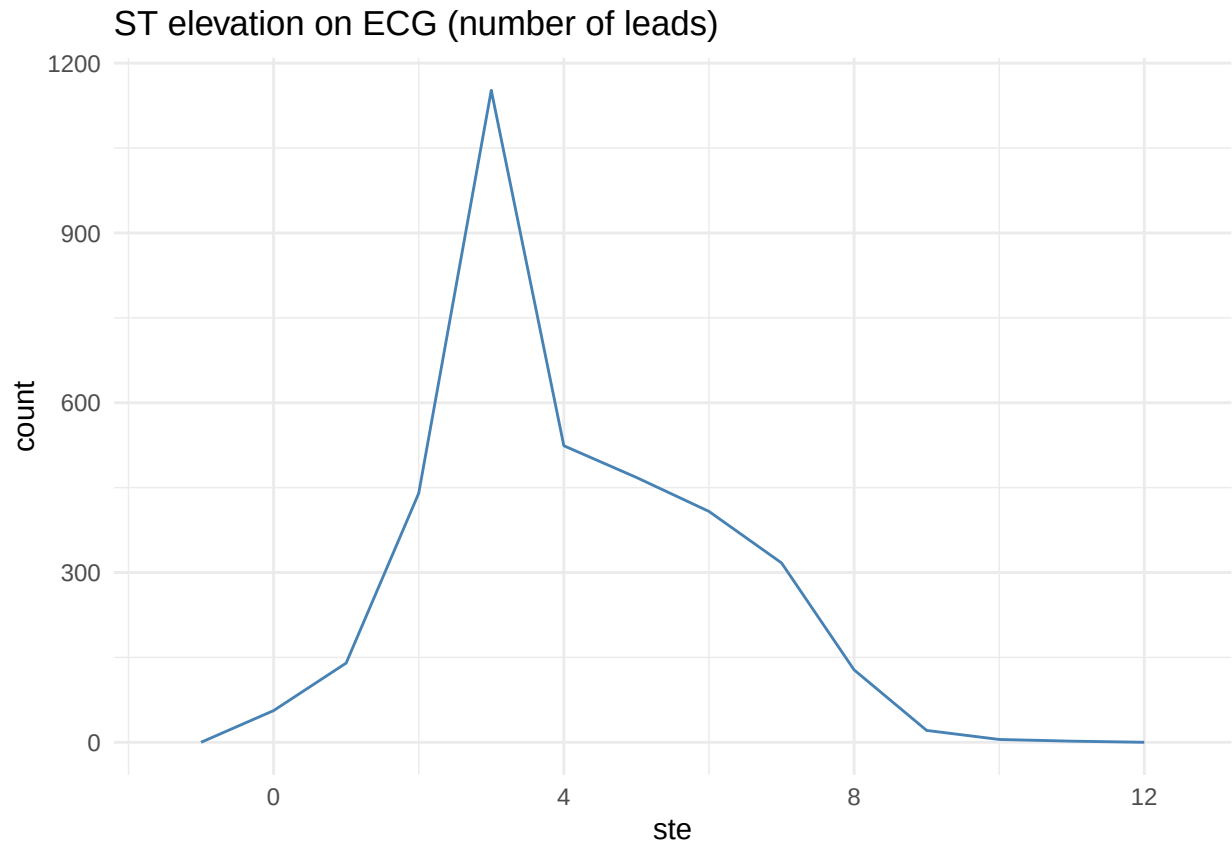
```
ami_data %>%
  count(killip, day30) %>%
  group_by(killip) %>%
  mutate(pct = n / sum(n)) %>%
  filter(day30 == "Dead") %>%
  ggplot(aes(x = killip, y = pct, fill = killip)) +
  geom_col(colour = "white", width = 0.7) +
  geom_text(aes(label = scales::percent(pct, accuracy = 0.1)),
            vjust = -0.4, size = 3.8, fontface = "bold") +
  scale_y_continuous(labels = scales::percent, limits = c(0, 0.65)) +
  scale_fill_brewer(palette = "Reds") +
  labs(title = "30-day mortality rate by Killip class",
       x = "Killip class", y = "Mortality rate") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "none", plot.title = element_text(face = "bold"))
```



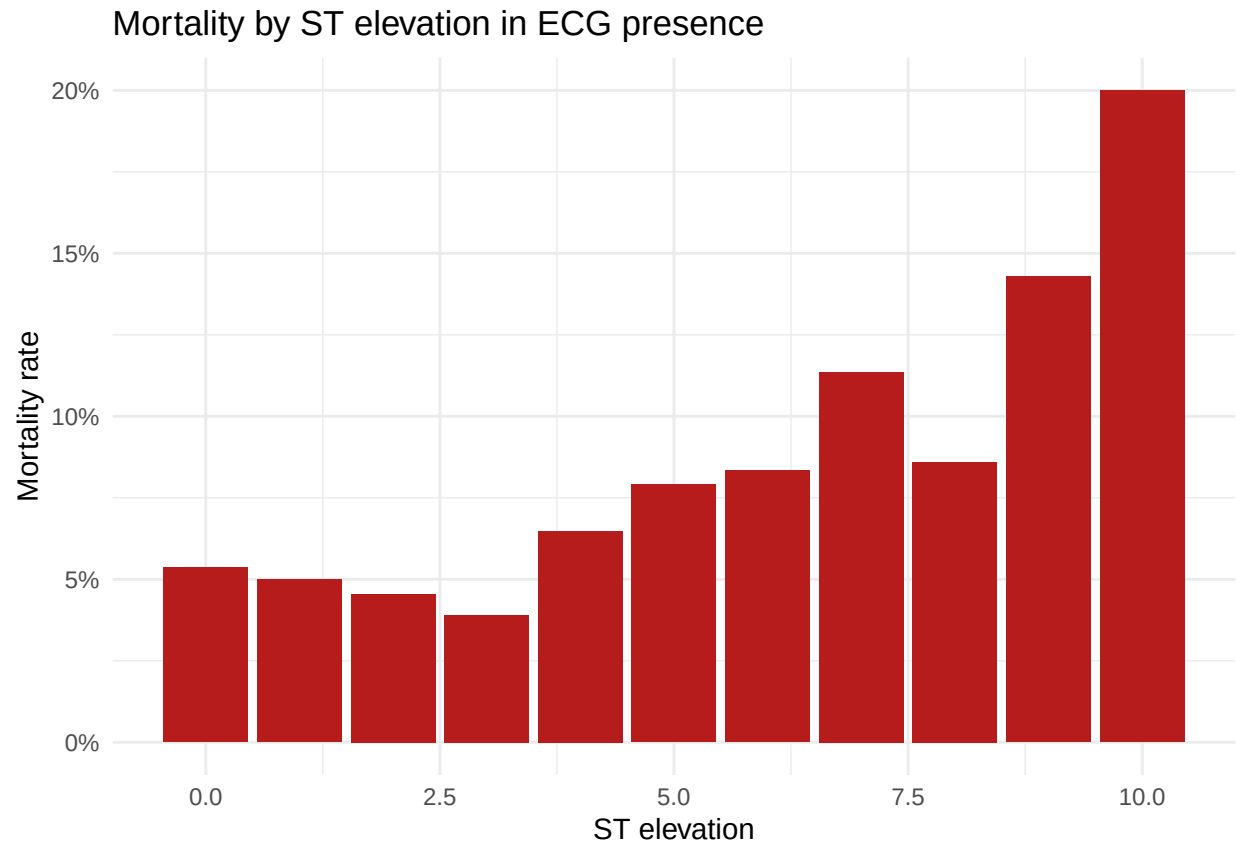
ST elevation

Greater ST segment elevation is clearly associated with a higher probability of death, suggesting that more extensive myocardial involvement worsens an acute prognosis.

```
ggplot(ami_data, aes(x = ste)) +
  geom_freqpoly(binwidth = 1, color = "steelblue") +
  labs(title = "ST elevation on ECG (number of leads)") + theme_minimal()
```



```
ami_data %>%  
  group_by(ste, day30) %>%  
  summarise(n = n(), .groups = "drop") %>%  
  group_by(ste) %>%  
  mutate(pct = n / sum(n)) %>%  
  filter(day30 == "Dead") %>%  
  ggplot(aes(x = ste, y = pct)) +  
  geom_col(fill = "#B71C1C") +  
  scale_y_continuous(labels = scales::percent) +  
  labs(title = "Mortality by ST elevation in ECG presence",  
       x = "ST elevation", y = "Mortality rate") +  
  theme_bw(base_size = 11) + theme_minimal()
```

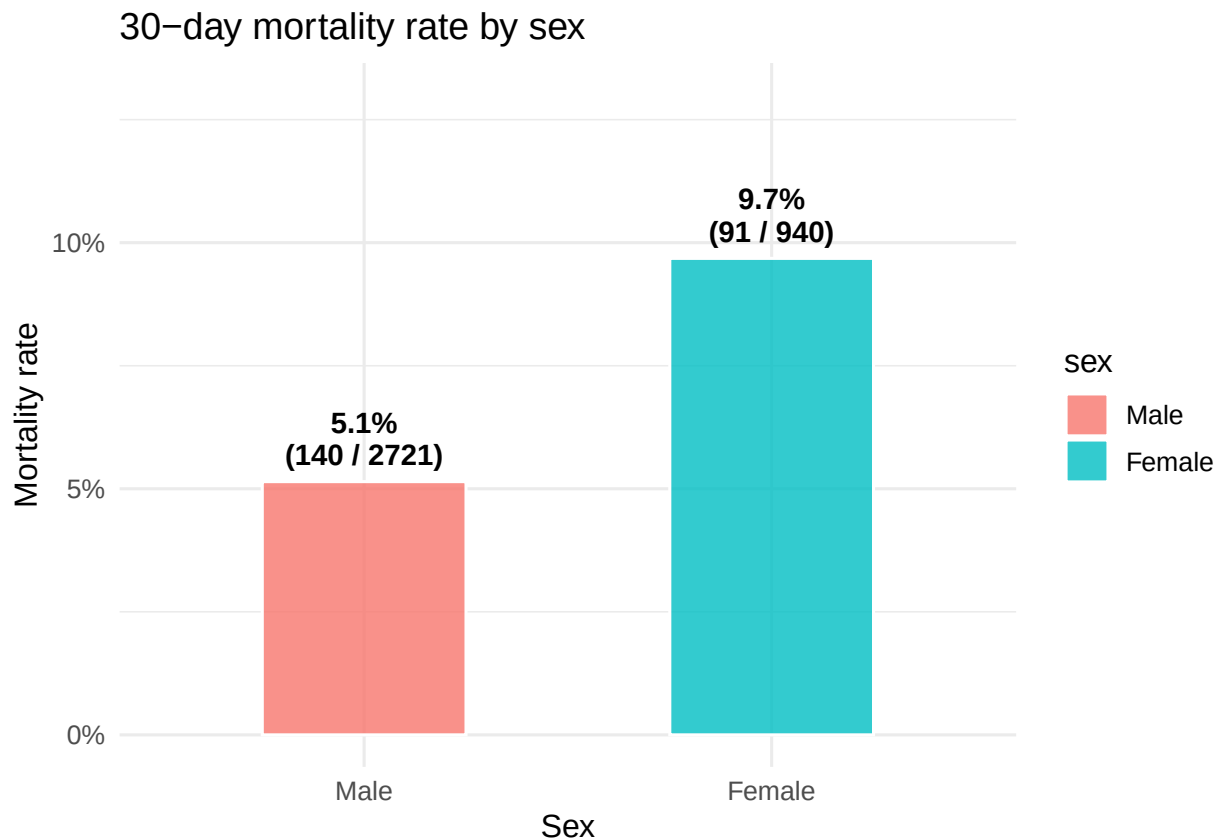


Sex

Female patients show a higher raw proportion of 30-day mortality compared to male patients.

```
ami_data %>%
  group_by(sex) %>%
  summarise(
    Deaths = sum(day30 == "Dead"),
    Total = n(),
    Mortality_Rate = Deaths / Total,
    .groups = "drop"
  ) %>%
  ggplot(aes(x = sex, y = Mortality_Rate, fill = sex)) +
  geom_col(width = 0.5, colour = "white", alpha = 0.8) +
  geom_text(
    aes(label = sprintf("%.1f%%\n(%d / %d)", Mortality_Rate * 100, Deaths, Total)),
    vjust = -0.3,
    lineheight = 0.9,
    fontface = "bold",
    size = 4
  ) +
  scale_y_continuous(labels = scales::percent, limits = c(0, 0.13)) +
  labs(
    title = "30-day mortality rate by sex",
    x = "Sex",
```

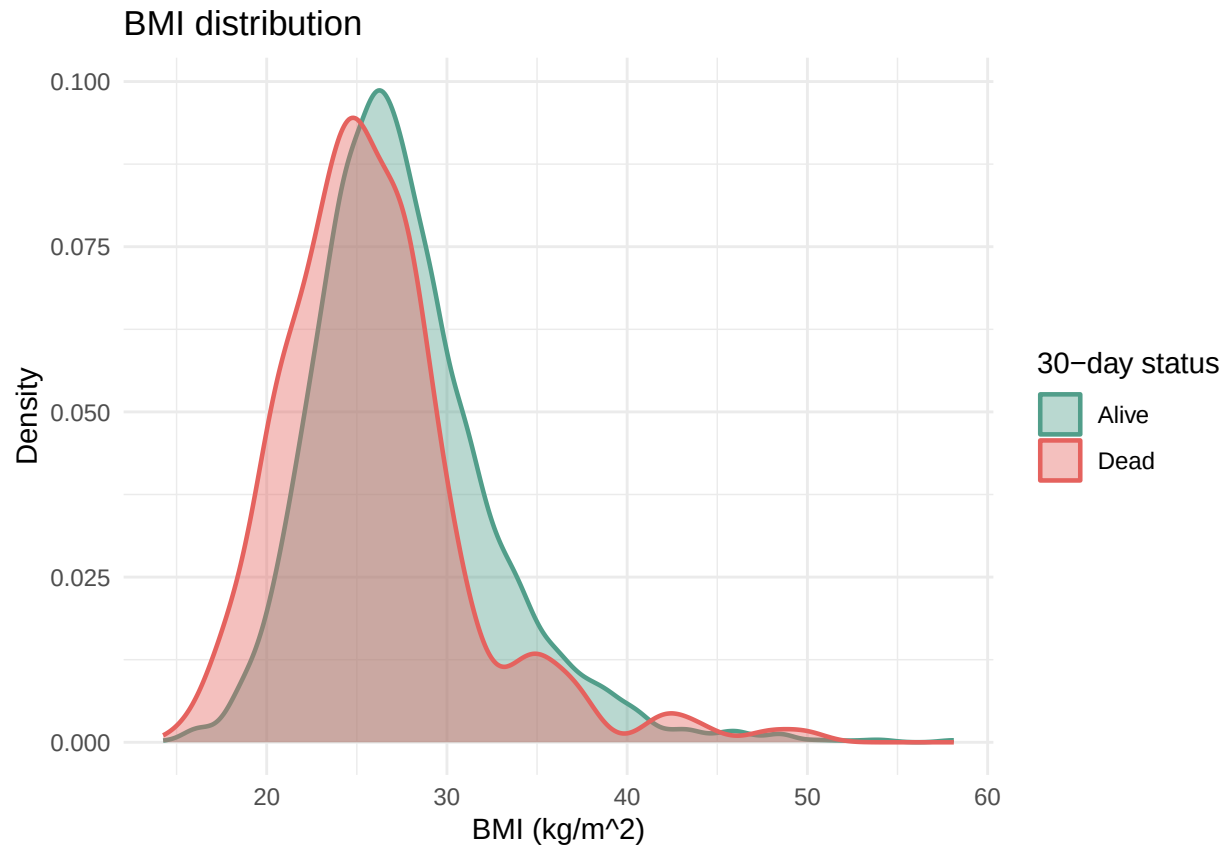
```
y = "Mortality rate"
) +
theme_minimal(base_size = 12)
```



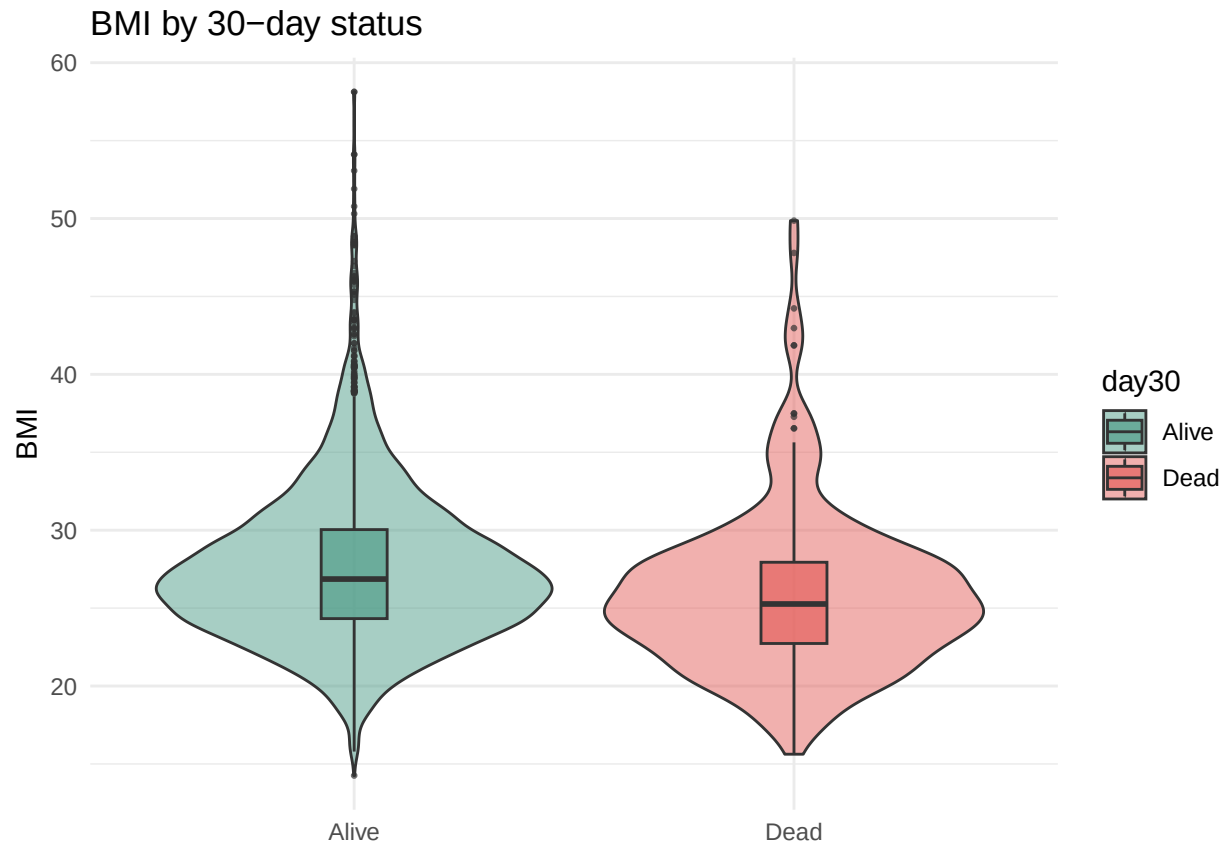
BMI

Patients who died tended to have lower BMI values.

```
ggplot(ami_data, aes(x = bmi, fill = day30, colour = day30)) +
  geom_density(alpha = 0.4, linewidth = 0.8) +
  scale_fill_manual(values = status_colors) +
  scale_colour_manual(values = status_colors) +
  labs(title = "BMI distribution", x = "BMI (kg/m^2)", y = "Density",
       fill = "30-day status", colour = "30-day status") +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom", plot.title = element_text(face = "bold")) + theme_minimal()
```



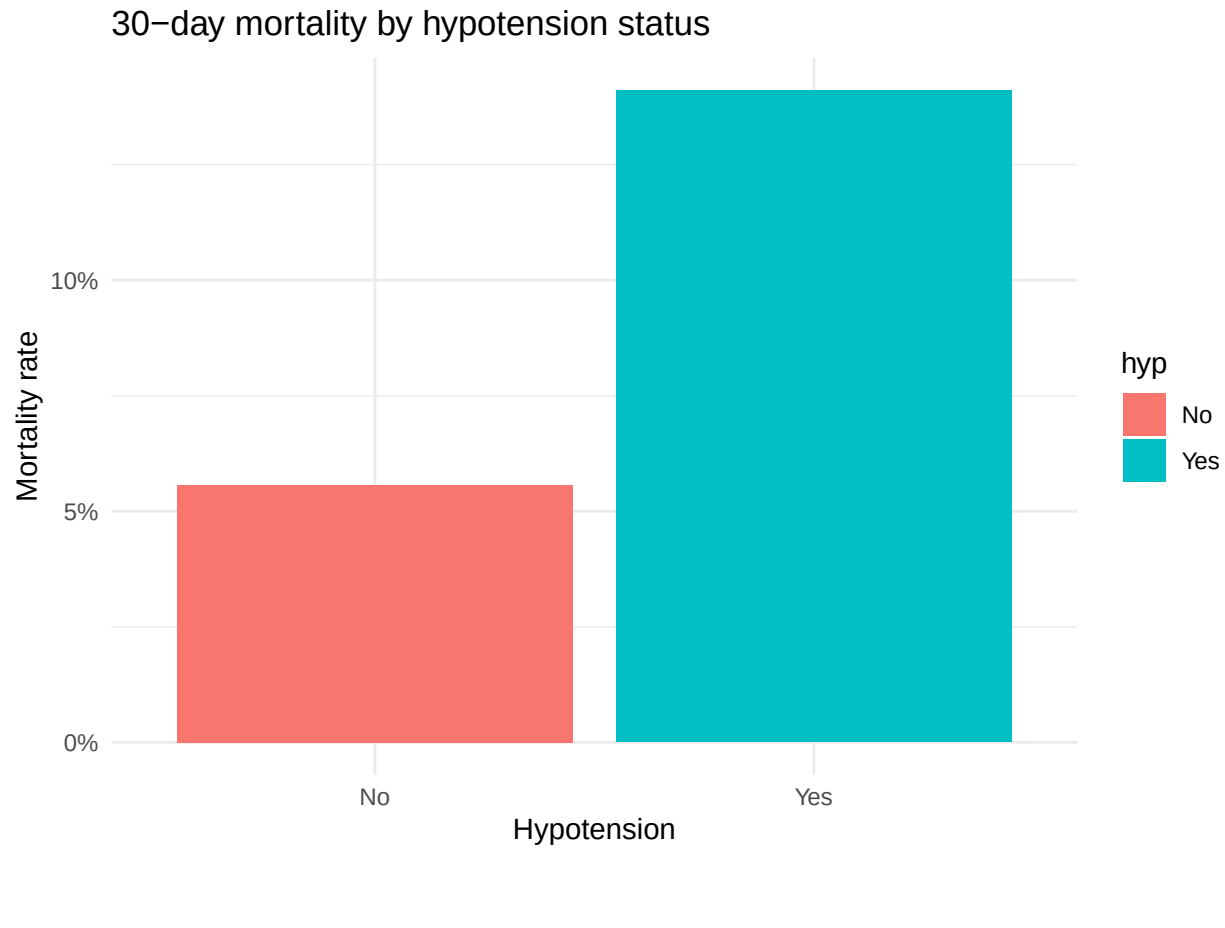
```
ggplot(ami_data, aes(x = day30, y = bmi, fill = day30)) +  
  geom_violin(alpha = 0.5, trim = TRUE) +  
  geom_boxplot(width = 0.15, outlier.size = 0.5, alpha = 0.7) +  
  scale_fill_manual(values = status_colors) +  
  labs(title = "BMI by 30-day status", x = NULL, y = "BMI") +  
  theme_minimal(base_size = 11) +  
  theme(legend.position = "none") + theme_minimal()
```



Hypotension

Patients presenting with hypotension had substantially higher mortality rates.

```
ami_data %>%
  count(hyp, day30) %>%
  group_by(hyp) %>%
  mutate(rate = n / sum(n)) %>%
  filter(day30 == "Dead") %>%
  ggplot(aes(x = hyp, y = rate, fill = hyp)) +
  geom_col() +
  scale_y_continuous(labels = scales::percent) +
  theme_minimal() +
  labs(
    title = "30-day mortality by hypotension status",
    x = "Hypotension",
    y = "Mortality rate"
  )
```



Question 2

Perform univariable regression analyses (crude association measures) of all candidate predictors for your prognostic model. For the continuous variables is the linear effect reasonable?

Crude odds and univariable analysis

The table below displays the crude associations between each predictor and 30-day mortality.

- **Strongest Predictors: Killip Class IV** exhibits the highest point estimate, increasing the odds of death nearly 29-fold (**OR = 28.91**, 95% CI: 11.5–75.6) compared to Class I. This is mirrored by the **Shock** variable (**OR = 15.8**). **Hypotension** nearly triples the risk (**OR = 2.79**), and **Tachycardia** more than doubles it (**OR = 2.39**).
- **Anterior infarct location** more than doubles mortality odds (**OR = 2.43**). For **ST elevation**, every additional lead increases the odds of death by 20% (**OR = 1.20**), while having > 4 leads elevated (**st4**) doubles the risk (**OR = 2.01**).
- **Age** increases the odds of death by 10% per year (**OR = 1.10**). **Female sex** (**OR = 1.98**), **Prior MI** (**OR = 2.24**), and **Current smoking** (**OR = 2.23**) all roughly double the crude odds of mortality. **BMI** shows an inverse relationship (**OR = 0.92**), indicating that a lower body mass is associated with higher acute mortality.

```
tbl_uvregression(
  ami_data,
  label = pretty_labels,
  method = glm,
  y = day30,
  method.args = list(family = binomial),
  exponentiate = TRUE,
  include = c(age, killip, sho, ant, hrt, ste, st4, smk, hyp, pmi, sex, bmi, pan, ttr, dia, fam, htn, .)
) %>% bold_labels()
```

Evaluating the linearity assumption for continuous variables

To determine if a linear effect is reasonable for our continuous predictors, we examine how the risk behaves across their ranges on the log-odds scale. We formally test this assumption by comparing a standard linear model against a non-linear model utilizing restricted cubic splines with 3 knots (`rcs(..., 3)`). A Likelihood Ratio Test (LRT) is then applied to assess whether the spline model offers a statistically significant improvement in fit.

Age, ST Elevation, and BMI are the only true continuous variables in our dataset. The other variables are binary or categorical and do not require this linearity assessment.

```
dd <- datadist(ami_data)
options(datadist = "dd")

check_linearity <- function(var, label) {
  f_lin <- glm(day30 ~ get(var), data = ami_data, family = binomial)
  f_spl <- glm(day30 ~ rcs(get(var), 3), data = ami_data, family = binomial)
  p_val <- anova(f_lin, f_spl, test = "LRT")$`Pr(>Chi)`[2]
  cat(sprintf("%-6s - LRT linearity p = %.3f  %s\n",
              label, p_val,
              ifelse(p_val < 0.05, "non-linear", "linear OK")))
}

check_linearity("age", "Age")
```

```
## Age      - LRT linearity p = 0.007  non-linear
```

```
check_linearity("ste", "ST elevation")
```

```
## ST elevation - LRT linearity p = 0.809  linear OK
```

```
check_linearity("bmi", "BMI")
```

```
## BMI      - LRT linearity p = 0.000  non-linear
```

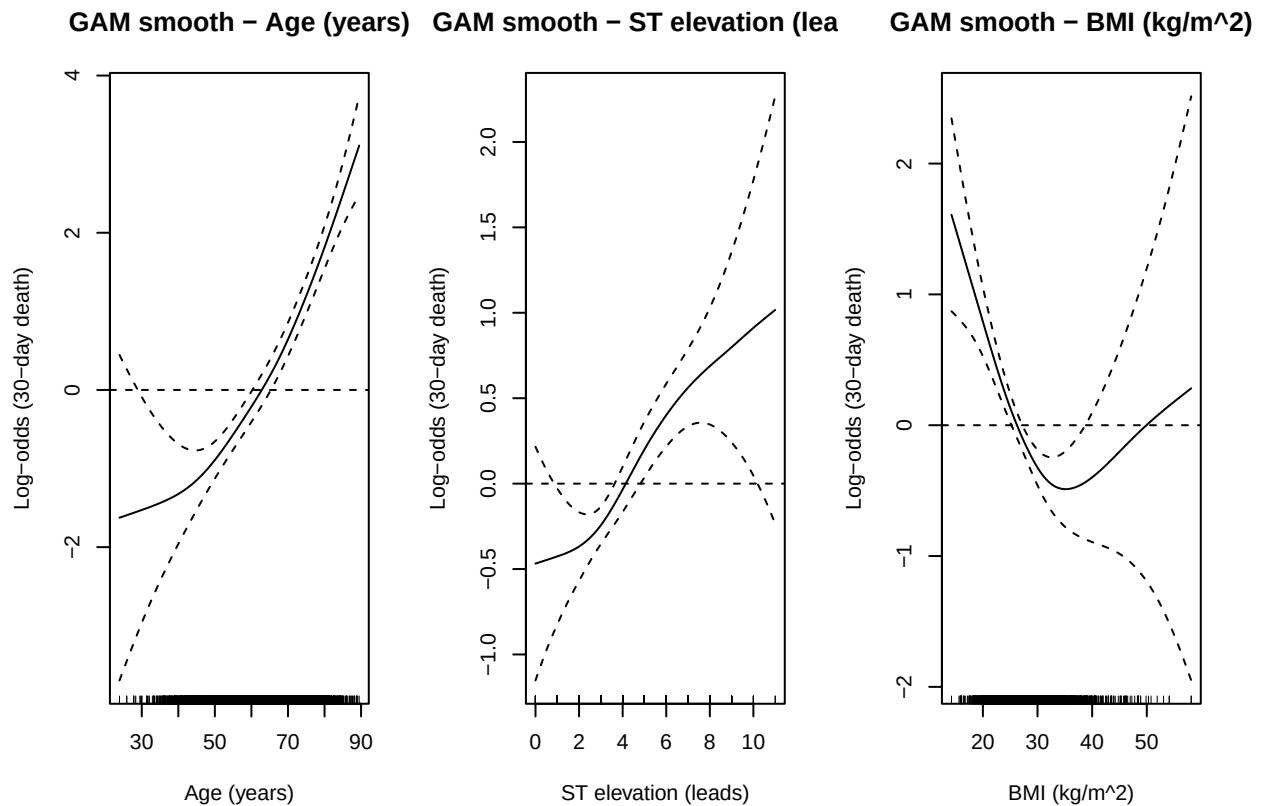
- Age ($p = 0.007$): The LRT rejects strict linearity.
- ST Elevation ($p = 0.809$): The non-significant p-value confirms the linearity assumption.
- BMI ($p < 0.001$): Linearity is strongly rejected.

GAM Plots

To visually inspect the log-linear relationship assumed by logistic regression, we fit a Generalized Additive Model (GAM) using a penalized regression spline (`s()`) for each continuous variable.

```
plot_linearity <- function(var, xlab) {
  fit <- gam(as.numeric(day30) ~ 1 ~ s(get(var)), data = ami_data, family = binomial)
  plot(fit, se = TRUE, rug = TRUE,
       xlab = xlab, ylab = "Log-odds (30-day death)",
       main = paste("GAM smooth -", xlab))
  abline(h = 0, lty = 2)
}

par(mfrow = c(1, 3))
plot_linearity("age", "Age (years)")
plot_linearity("ste", "ST elevation (leads)")
plot_linearity("bmi", "BMI (kg/m^2)")
```



```
par(mfrow = c(1, 1))
```

The lines represent the estimated smooth effect on the log-odds of 30-day mortality, the dashed lines indicate the 95% confidence intervals, and the bottom displays the individual patient observations.

- **Age** The relationship is clearly curvilinear. The risk curve rises gradually from around age 25, then accelerates sharply upward from age 40–45, with the log-odds of death crossing zero around age 55–60. This monotonically increasing, non-linear pattern means that forcing a single straight line would

underestimate mortality risk in elderly patients and overestimate it in younger ones. This justifies the use of a spline transformation.

- **ST** The curve follows a linear upward trend across the most populated range of the data (0–8 leads), with each additional lead associated with a fairly steady increase in the log-odds of death. A slight flattening is visible beyond 6–8 leads, but the confidence intervals widen considerably in that region due to sparse data, making the tail unreliable. Given that the LRT fails to reject linearity ($p = 0.809$), a linear term is accepted as a reasonable approximation for this variable.
 - **BMI** The log-odds of death decrease steeply as BMI rises from the lowest values, reaching minimum around 30 kg/m^2 . Beyond 30 kg/m^2 , the curve gradually turns upward. However, the confidence intervals become very wide past approximately 35 kg/m^2 , reflecting the small number of patients in that range and making the upturn statistically uncertain. The non-linear shape justifies modeling BMI with a spline transformation.
-

Question 3

Build a multivariable prognostic model:

As you know, various selection procedures of the candidate predictors could be performed (try different approaches and compare them... we don't have here the expert's opinion, therefore the final choice will be based only on statistical evaluations with the aim of maximising predictive accuracy).

Pay attention to the multicollinearity between predictors (and to use the same variables but recoded in different ways)

Preprocessing

To begin our modelling, we first transform the Killip variable to two different variables: `killip2` and `killip3`.

- **Killip2** is a binary variable: 0 for a Killip score of 1, and 1 for Killip scores of 2, 3, and 4.
- **Killip3** is an ordinal variable: a Killip score of 1 maps to 3, a score of 2 maps to 2, and scores of 3 and 4 map to 1.

```
data <- data %>%
  mutate(killip2 = case_when(
    killip == 1 ~ 0,
    killip %in% c(2, 3, 4) ~ 1
  ))

data <- data %>%
  mutate(killip3 = case_when(
    killip == 1 ~ 3,
    killip == 2 ~ 2,
    killip %in% c(3, 4) ~ 1
  ))
```

The aim of this study is to build a predictive prognostic model that estimates the probability of dying within the first 30 days after a myocardial infarction, considering several candidate predictors. Predictive models summarise the effects of multiple predictors to provide more individualised risk estimates.

To begin, the dataset is split into 70% training and 30% test sets. This split allows us to train and tune the model on the training set, and then evaluate its generalisability on unseen data.

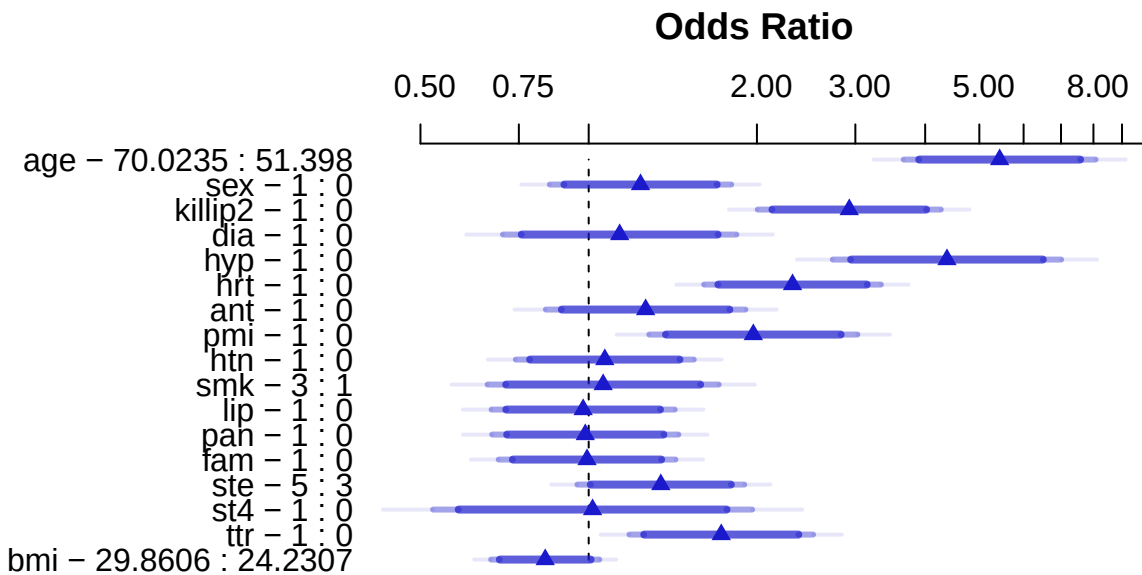
```
set.seed(123)
trainIndex <- createDataPartition(data$day30, p = 0.7, list = FALSE)
train <- data[trainIndex, ]
test <- data[-trainIndex, ]
dd <- datadist(train)
options(datadist = "dd")
```

Full Models

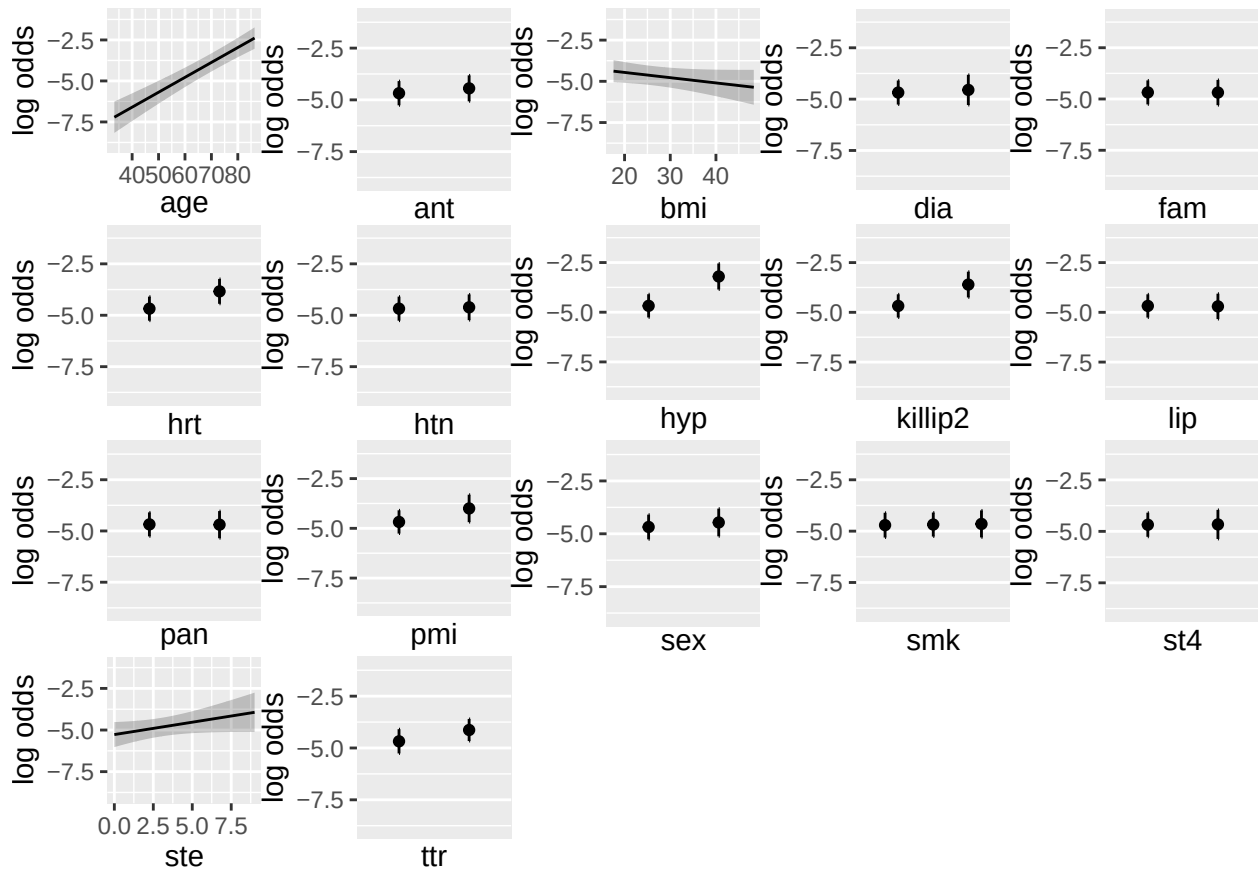
Full Linear Model

A full model including all candidate variables is fitted using standard logistic regression. The `plot(summary(model1))` function plots the inter-quartile range odds ratios; the bars represent confidence intervals. If a confidence interval includes 1, the variable contributes little information to the model.

```
model1 <- lrm(day30 ~ age + sex + killip2 + dia + hyp + hrt + ant + pmi + htn + smk + lip + pan + fam +  
plot(summary(model1), log = TRUE)
```



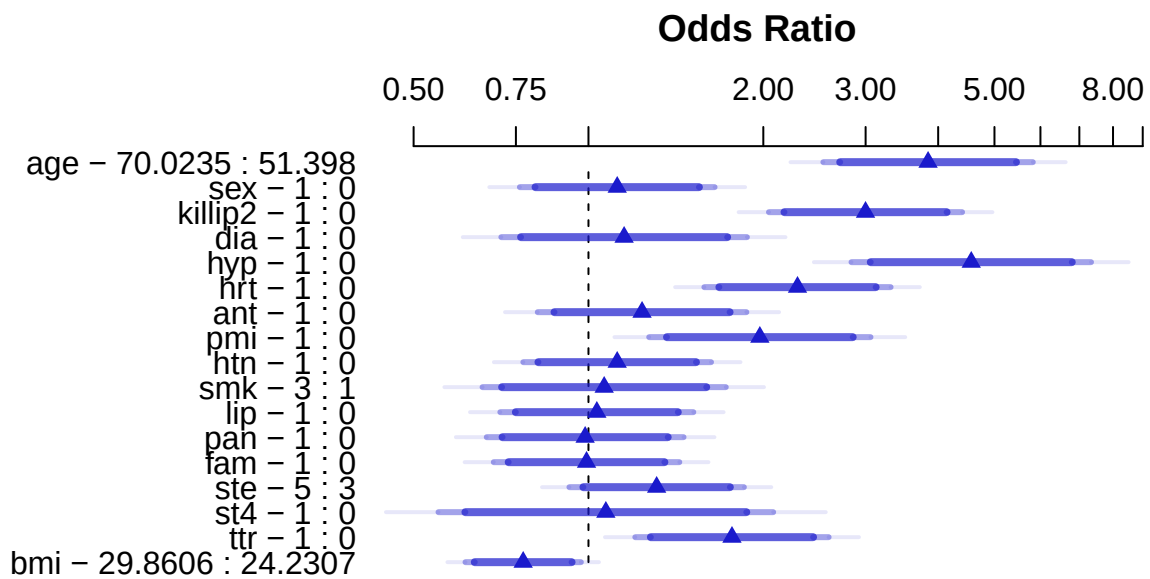
```
ggplot(Predict(model1))
```



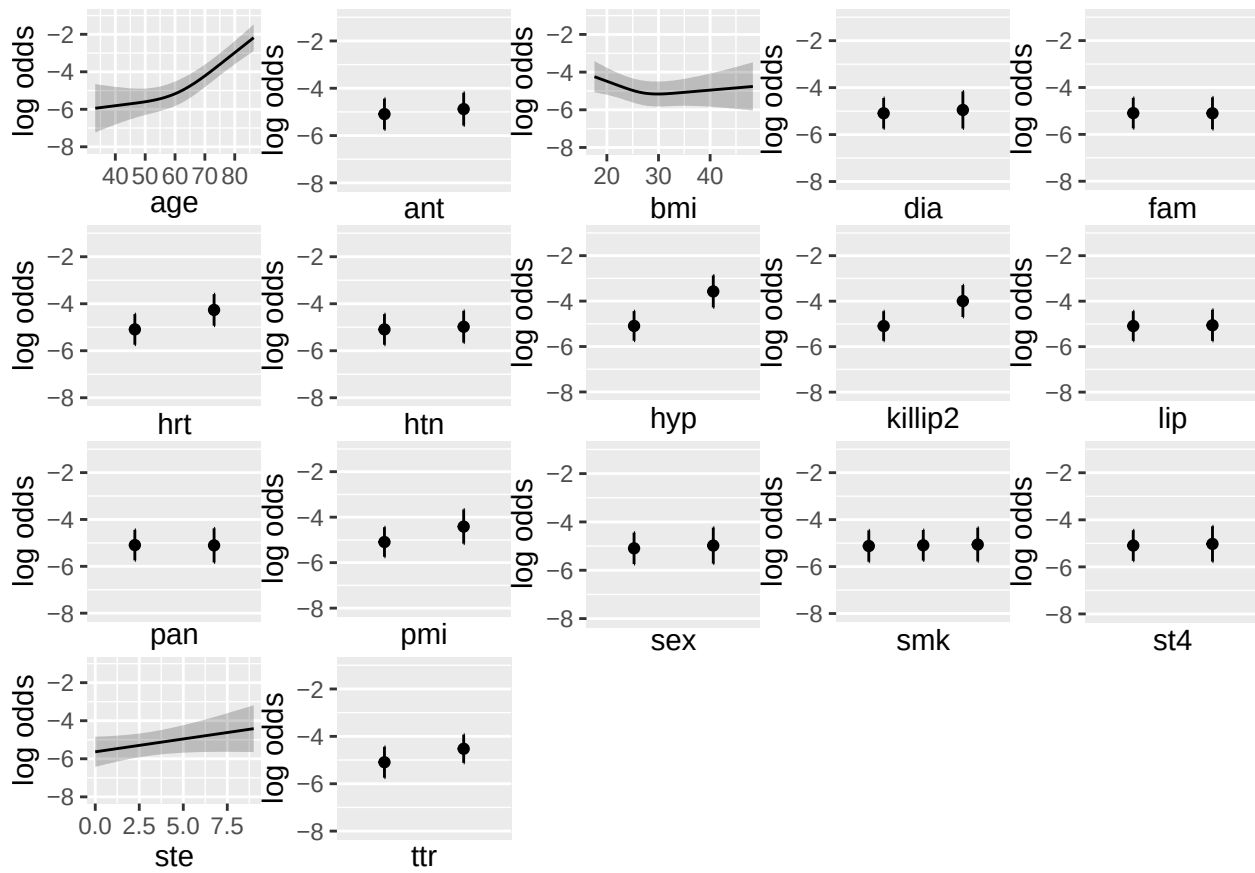
Full Model with Splines

Spline transformations are applied to `age` and `bmi`, since both show a non-linear relationship with the outcome `day30`.

```
model2 <- lrm(day30 ~ rcs(age,3) + sex + killip2 + dia + hyp + hrt + ant + pmi + htn + smk + lip + pan +
plot(summary(model2), log = TRUE)
```



```
ggplot(Predict(model2))
```



Variable Selection

To avoid overfitting, variable selection is needed to exclude the ones that have no effect on the dependent variable. In order to do this, backward selection and lasso were chosen.

Backward Elimination

The backward selection is applied to both the linear and the spline full models. This method implements a step-wise removal of predictors from the full model that have the least statistical significance. Variables selected are those with p-value larger than a specific threshold. This threshold could be chosen either by a fix value chosen at random or Akaike Information Criterion (AIC) or Bayesian Information Criterion (BIC).

The `fastbw` function in R uses an AIC-based threshold to iteratively remove predictors with the least statistical contribution. It prints deletion statistics at each step and approximate parameter estimates for the reduced model.

```
fastbw(model1)
```

```
##
## Deleted Chi-Sq d.f. P      Residual d.f. P      AIC
## fam      0.00  1    0.9709  0.00    1    0.9709  -2.00
## st4      0.00  1    0.9601  0.00    2    0.9981  -4.00
## pan      0.01  1    0.9417  0.01    3    0.9998  -5.99
## lip      0.02  1    0.8974  0.03    4    0.9999  -7.97
```

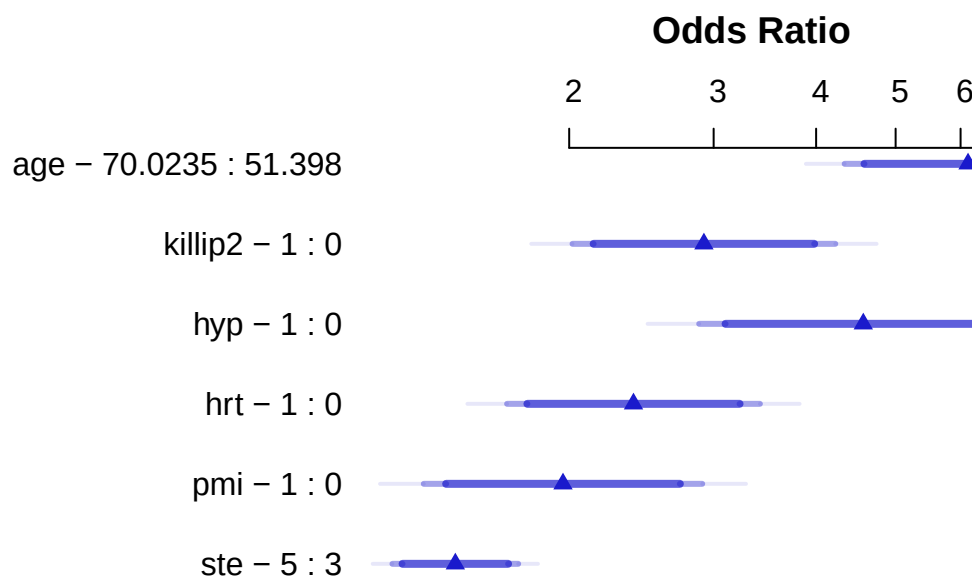
```
##      smk      0.06    1    0.8130  0.08      5  0.9999 -9.92
##      htn      0.14    1    0.7131  0.22      6  0.9998 -11.78
##      dia      0.32    1    0.5698  0.54      7  0.9993 -13.46
##      ant      1.28    1    0.2586  1.82      8  0.9861 -14.18
##      sex      1.28    1    0.2578  3.10      9  0.9603 -14.90
##      bmi      2.14    1    0.1436  5.24     10  0.8748 -14.76
##      ttr      8.09    1    0.0045 13.32     11  0.2728 -8.68
##
## Approximate Estimates after Deleting Factors
##
##              Coef      S.E.  Wald Z      P
## Intercept -10.75634 0.737114 -14.593 0.000e+00
## age        0.09651 0.009518  10.140 0.000e+00
## killip2    1.05951 0.189827   5.581 2.385e-08
## hyp        1.49747 0.237576   6.303 2.917e-10
## hrt        0.86678 0.182346   4.754 1.999e-06
## pmi        0.66511 0.201240   3.305 9.496e-04
## ste        0.18480 0.045154   4.093 4.265e-05
##
## Factors in Final Model
##
## [1] age      killip2 hyp      hrt      pmi      ste
```

```
fastbw(model2)
```

```
##
## Deleted Chi-Sq d.f. P      Residual d.f. P      AIC
## fam      0.00    1    0.9675  0.00      1  0.9675 -2.00
## pan      0.00    1    0.9446  0.01      2  0.9968 -3.99
## lip      0.03    1    0.8716  0.03      3  0.9985 -5.97
## st4      0.04    1    0.8361  0.08      4  0.9993 -7.92
## smk      0.06    1    0.8053  0.14      5  0.9997 -9.86
## dia      0.32    1    0.5701  0.46      6  0.9983 -11.54
## sex      0.38    1    0.5393  0.84      7  0.9971 -13.16
## htn      0.65    1    0.4195  1.49      8  0.9929 -14.51
## ant      0.94    1    0.3319  2.43      9  0.9827 -15.57
## bmi      6.01    2    0.0495  8.44     11  0.6734 -13.56
## ttr      8.17    1    0.0043 16.61     12  0.1647 -7.39
##
## Approximate Estimates after Deleting Factors
##
##              Coef      S.E.  Wald Z      P
## Intercept -6.78770 1.47845 -4.5911 4.410e-06
## age        0.02342 0.02635  0.8886 3.742e-01
## age'       0.06913 0.02487  2.7795 5.444e-03
## killip2    1.07300 0.19271  5.5680 2.577e-08
## hyp        1.54610 0.23998  6.4426 1.174e-10
## hrt        0.85190 0.18456  4.6159 3.914e-06
## pmi        0.70750 0.20569  3.4397 5.823e-04
## ste        0.18109 0.04551  3.9791 6.919e-05
##
## Factors in Final Model
##
## [1] age      killip2 hyp      hrt      pmi      ste
```

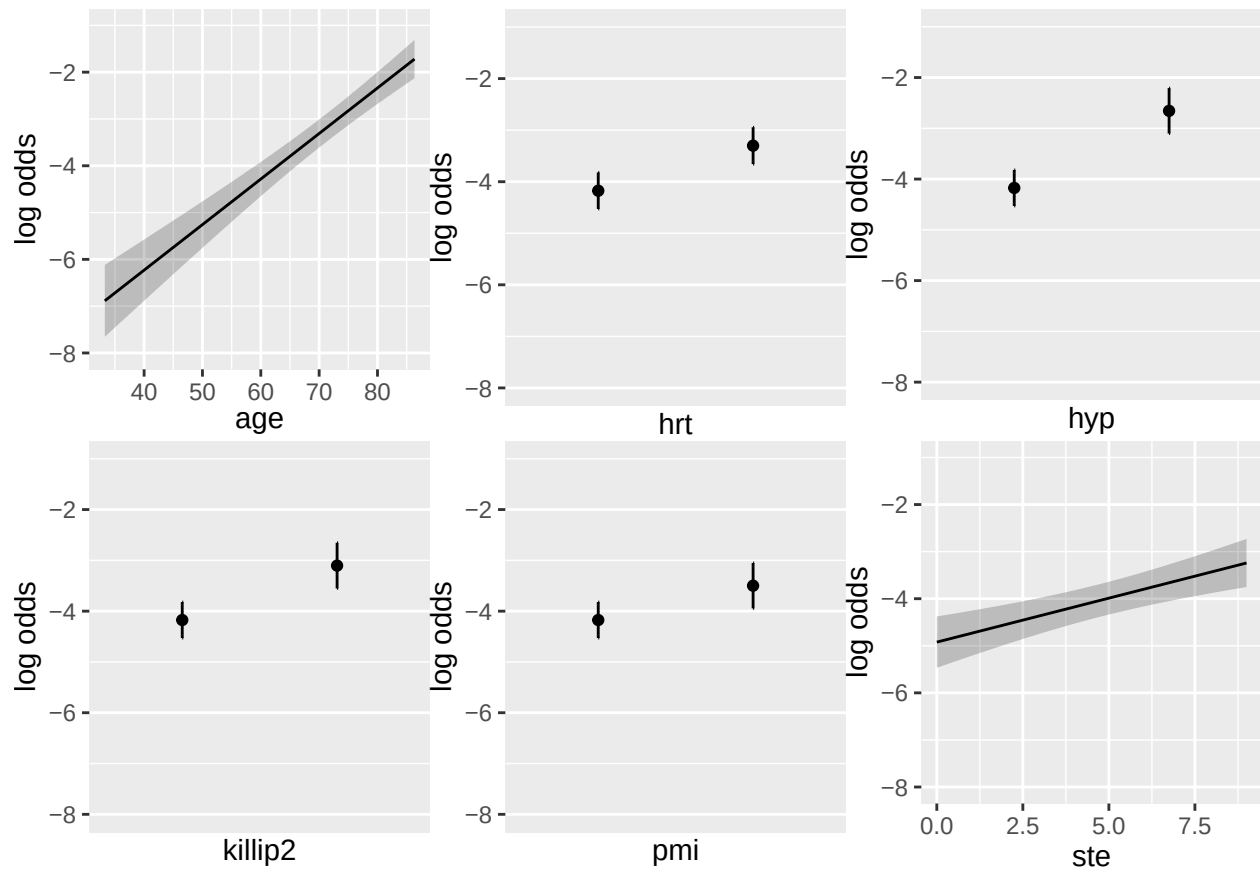
Both models yield the same selected variables: **age**, **killip2**, **hypotension**, **heart rate**, **previous myocardial infarction**, and **ST elevation on the ECG**.

```
model13 <- lrm(day30 ~ age + killip2 + hyp + hrt + pmi + ste, data = train, x = TRUE, y = TRUE)
plot(summary(model13), log = TRUE)
```



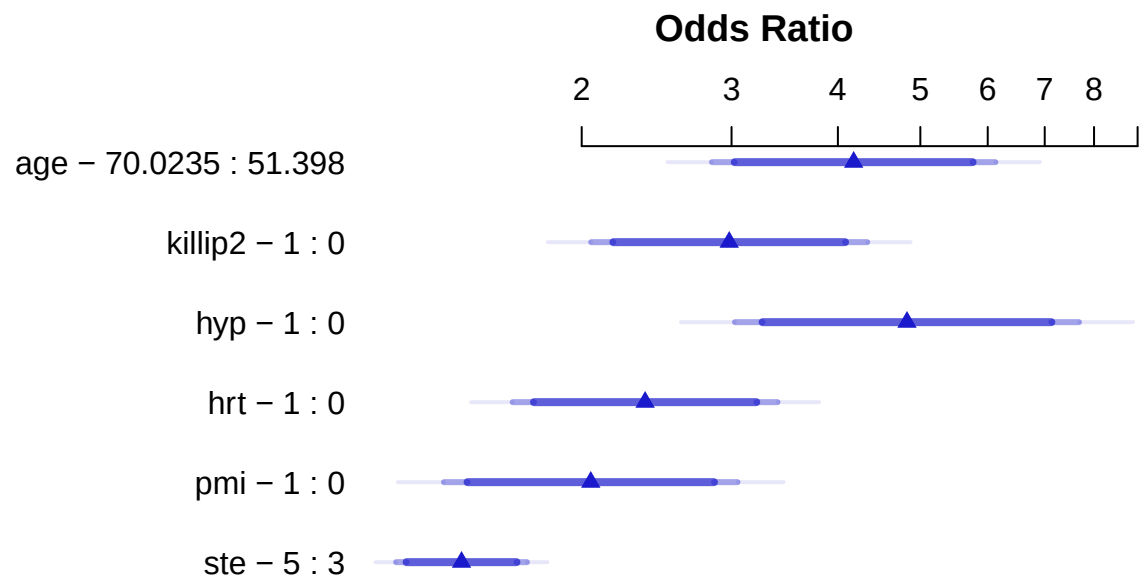
Backward-Selected Linear Model

```
ggplot(Predict(model13))
```

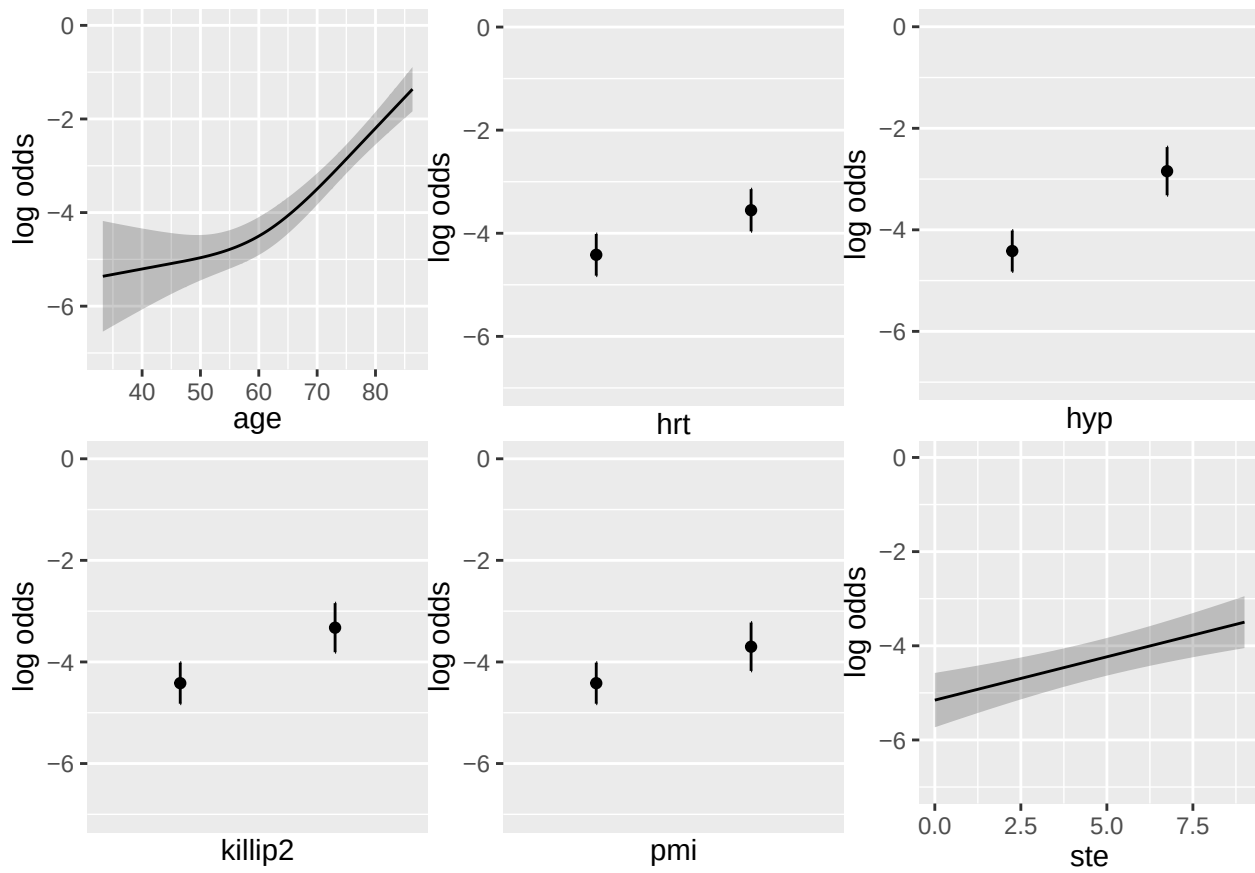


Backward-Selected Spline Model

```
model14 <- lrm(day30 ~ rcs(age,3) + killip2 + hyp + hrt + pmi + ste, data = train, x = TRUE, y = TRUE)
plot(summary(model14), log = TRUE)
```



```
ggplot(Predict(model14))
```



LASSO Penalisation

LASSO penalisation is applied to the full model to further address overfitting and multicollinearity. It adds a penalty term to the loss function, shrinking less important coefficients to zero and thus performing automatic variable selection. LASSO is applied to both the linear and the spline formulations.

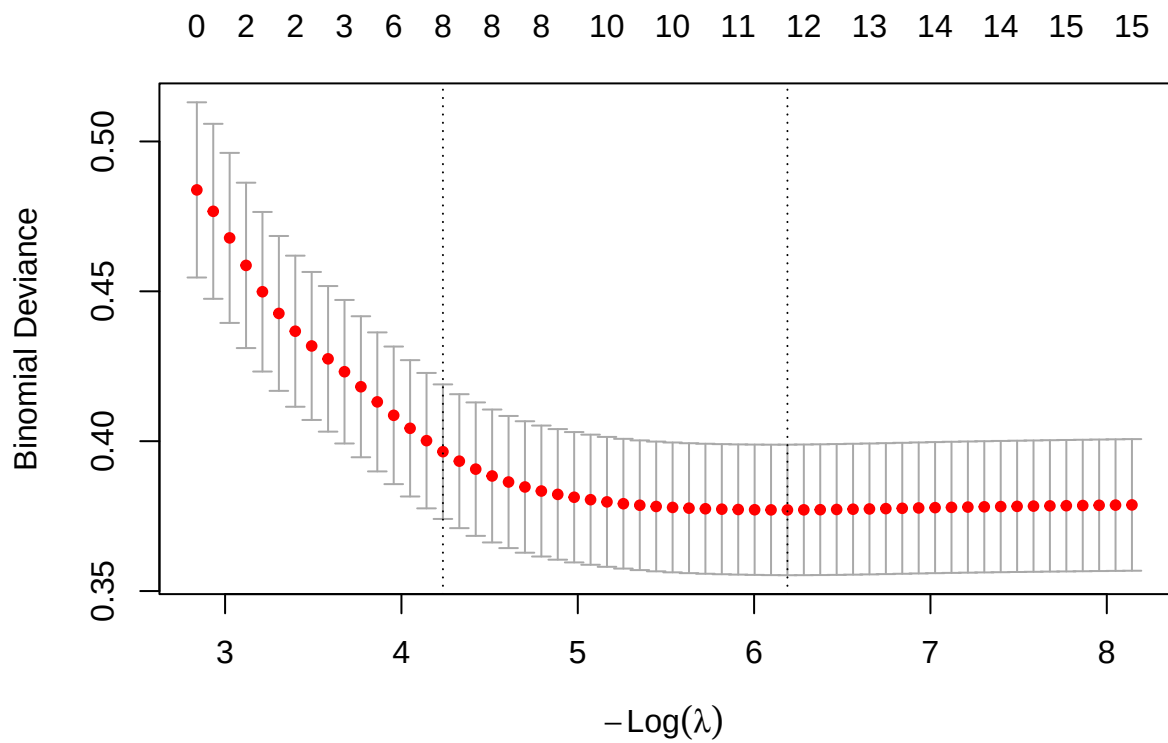
The linear LASSO additionally selects **anterior infarction location** and **time to relief of chest pain (ttr)** compared to backward selection.

```
predictors <- c("age", "sex", "killip2", "dia", "hyp", "hrt", "ant", "pmi", "htn", "smk", "lip", "pan")
X_train1 <- model.matrix(day30 ~ . - 1, data = train[, c("day30", predictors)])
Y_trainlasso <- train$day30
cv_lasso1 <- cv.glmnet(X_train1, Y_trainlasso, family = "binomial", alpha = 1)
print(coef(cv_lasso1, s = "lambda.1se"))
```

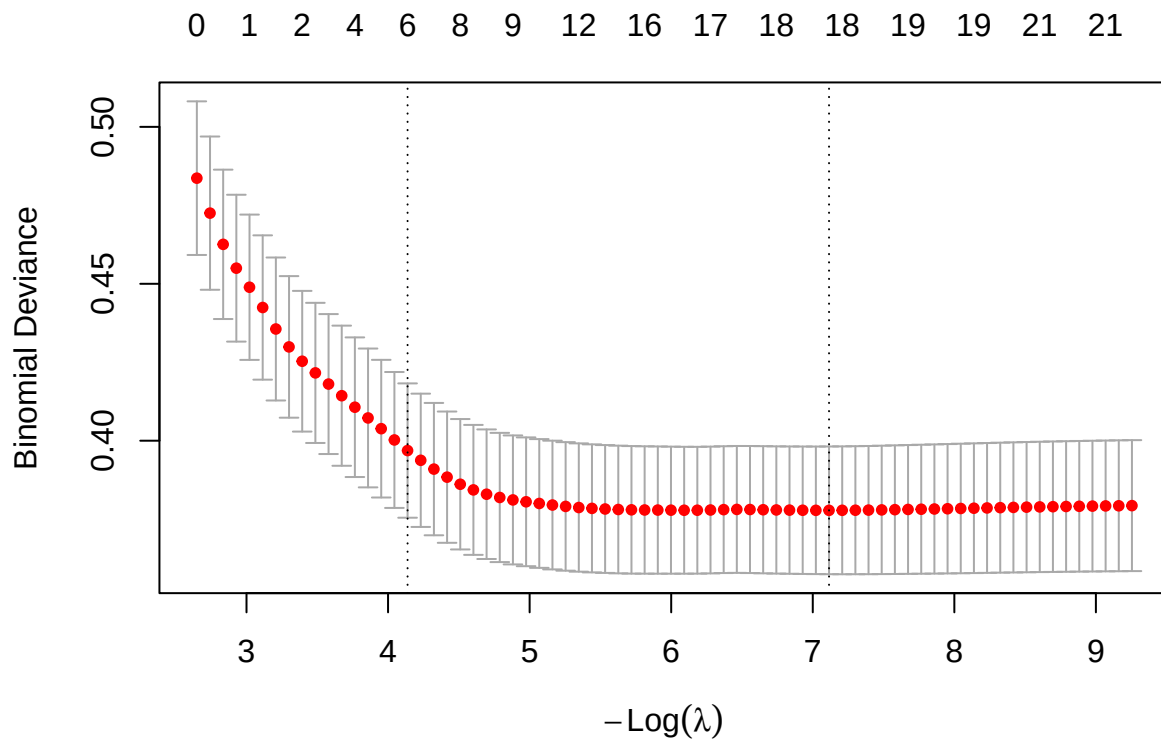
```
## 18 x 1 sparse Matrix of class "dgCMatrix"
##          lambda.1se
## (Intercept) -7.315336488
## age          0.062759965
## sex          .
## killip2      0.864275077
## dia          .
## hyp          0.687555280
## hrt          0.347201856
## ant          0.005108785
```

```
## pmi      0.162132950
## htn      .
## smk      .
## lip      .
## pan      .
## fam      .
## ste      0.043294729
## st4      .
## ttr      0.015227696
## bmi      .
```

```
plot(cv_lasso1)
```



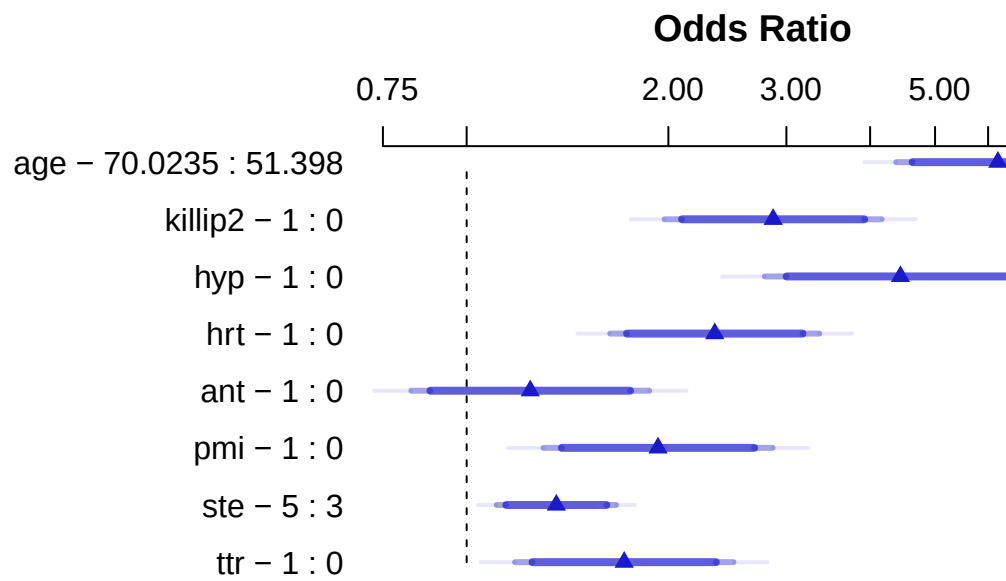
```
formulaS <- as.formula("day30 ~ ns(age, df = 3) + sex + killip2 + dia + hyp + hrt + ant + pmi + htn + s
X_train2 <- model.matrix(formulaS, data = train)[, -1]
cv_lasso2 <- cv.glmnet(X_train2, Y_trainlasso, family = "binomial", alpha = 1)
plot(cv_lasso2)
```



```
coefficients2 <- coef(cv_lasso2, s = "lambda.1se")
print(coefficients2[coefficients2[, 1] != 0, , drop = FALSE])
```

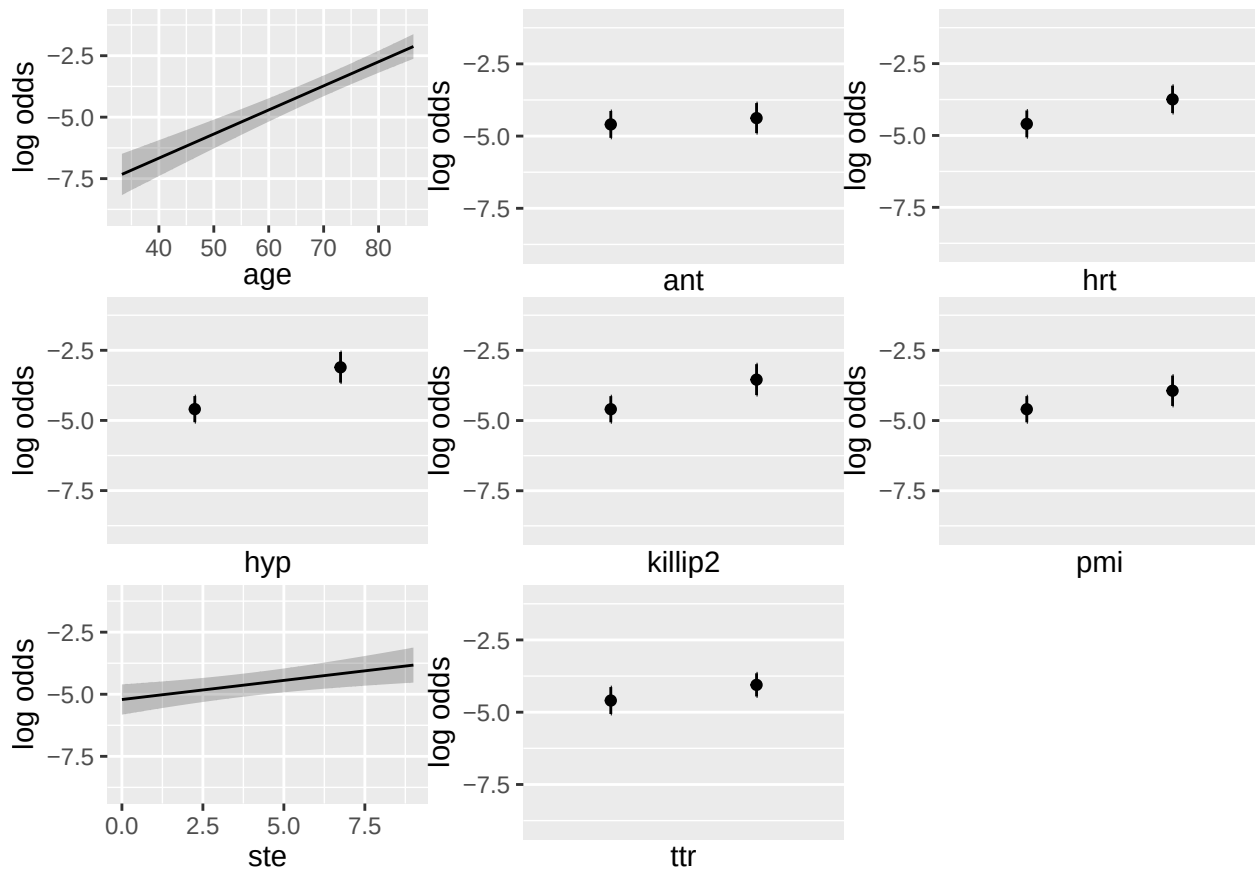
```
## 7 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.1se
## (Intercept)    -2.86007375
## ns(age, df = 3)3  2.90717144
## killip2         0.85564660
## hyp             0.69257839
## hrt             0.27122796
## pmi             0.14840578
## ste             0.02572345
```

```
model5 <- model_linear_lrm <- lrm(day30 ~ age + killip2 + hyp + hrt + ant+ pmi + ste + ttr, data = tra)
plot(summary(model5), log = TRUE)
```



LASSO-Selected Linear Model

```
ggplot(Predict(model5))
```

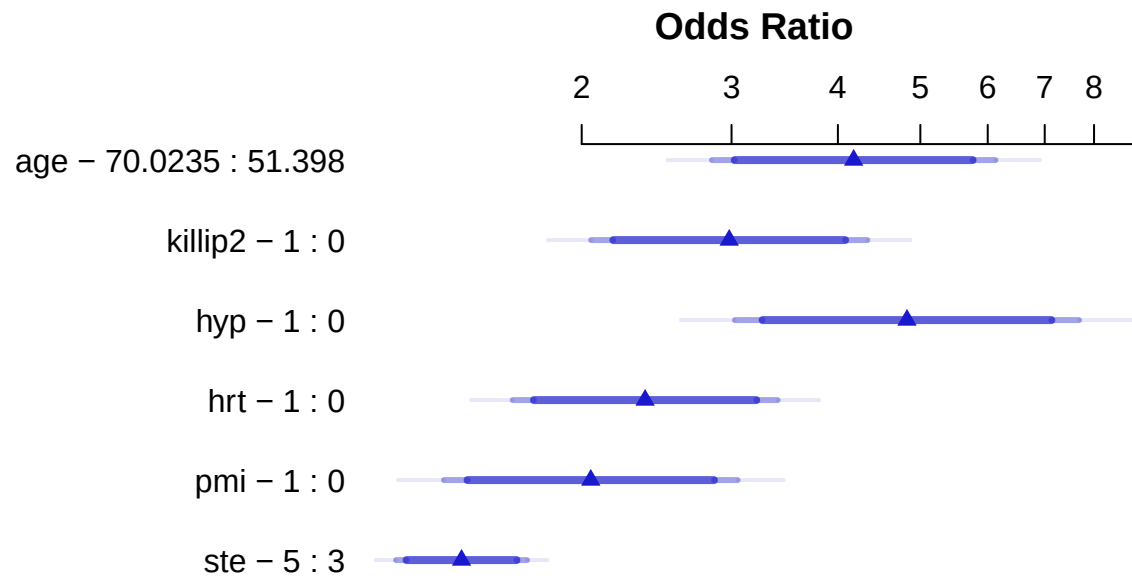


```
model6 <- lrm(day30 ~ rcs(age, 3) + killip2 + hyp + hrt + pmi + ste, data = train, x = TRUE, y = TRUE)
summary(model6)
```

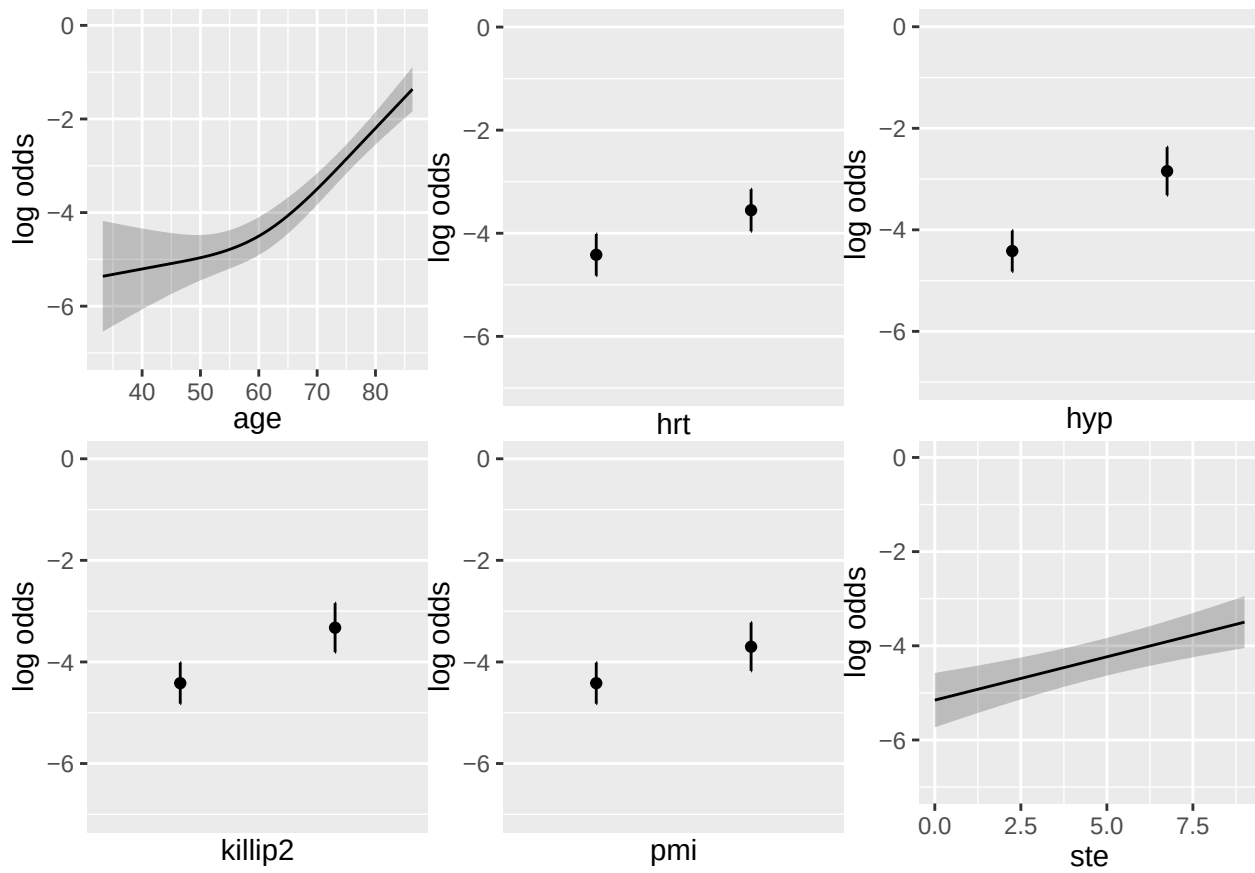
LASSO-Selected Spline Model

##		Effects			Response : day30				
##									
##	Factor	Low	High	Diff.	Effect	S.E.	Lower 0.95	Upper 0.95	
##	age	51.398	70.024	18.625	1.42900	0.19562	1.04560	1.81250	
##	Odds Ratio	51.398	70.024	18.625	4.17470	NA	2.84520	6.12550	
##	killip2	0.000	1.000	1.000	1.09230	0.19055	0.71885	1.46580	
##	Odds Ratio	0.000	1.000	1.000	2.98120	NA	2.05210	4.33090	
##	hyp	0.000	1.000	1.000	1.57330	0.23747	1.10780	2.03870	
##	Odds Ratio	0.000	1.000	1.000	4.82240	NA	3.02780	7.68060	
##	hrt	0.000	1.000	1.000	0.86475	0.18282	0.50643	1.22310	
##	Odds Ratio	0.000	1.000	1.000	2.37440	NA	1.65940	3.39760	
##	pmi	0.000	1.000	1.000	0.71770	0.20263	0.32055	1.11490	
##	Odds Ratio	0.000	1.000	1.000	2.04970	NA	1.37790	3.04910	
##	ste	3.000	5.000	2.000	0.36803	0.09032	0.19100	0.54505	
##	Odds Ratio	3.000	5.000	2.000	1.44490	NA	1.21050	1.72470	

```
plot(summary(model6), log = TRUE)
```



```
ggplot(Predict(model6))
```



At the end of this process we have **6 models**: 2 full models, 2 backward-selected models, and 2 LASSO-penalised models (each pair with and without splines).

Multicollinearity Check

Variance Inflation Factors (VIF) are computed for all models to assess multicollinearity among predictors.

```
vif(model1)
```

```
##      age      sex  killip2      dia      hyp      hrt      ant      pmi      htn      smk
## 1.345683 1.103791 1.059551 1.087475 1.091294 1.103852 1.416531 1.202960 1.115320 1.189392
##      lip      pan      fam      ste      st4      ttr      bmi
## 1.087949 1.193875 1.071144 3.862506 3.619918 1.022714 1.151591
```

```
vif(model2)
```

```
##      age      age'      sex  killip2      dia      hyp      hrt      ant      pmi
## 10.081416 10.159317 1.149109 1.064384 1.090274 1.096235 1.111526 1.404116 1.216561
##      htn      smk      lip      pan      fam      ste      st4      ttr      bmi
## 1.127057 1.213965 1.098578 1.205042 1.074434 3.859277 3.613885 1.025307 5.846322
##      bmi'
## 5.642927
```

```
vif(model3)
```

```
##      age killip2      hyp      hrt      pmi      ste
## 1.038783 1.033106 1.069335 1.064931 1.020969 1.025496
```

```
vif(model4)
```

```
##      age      age' killip2      hyp      hrt      pmi      ste
## 10.034551 10.059502 1.036433 1.078009 1.067015 1.028143 1.028657
```

```
vif(model5)
```

```
##      age killip2      hyp      hrt      ant      pmi      ste      ttr
## 1.048700 1.041719 1.073881 1.080282 1.406289 1.023379 1.374130 1.010658
```

```
vif(model6)
```

```
##      age      age' killip2      hyp      hrt      pmi      ste
## 10.034551 10.059502 1.036433 1.078009 1.067015 1.028143 1.028657
```

Question 4

Evaluate model's performance in terms of discrimination. Evaluate model's performance in terms of calibration.

In predictive modelling, evaluating accuracy is as important as interpreting coefficients. Two types of validation are distinguished:

- **Internal validation:** evaluation on the training dataset.
- **External validation:** evaluation on the held-out test dataset.

The final model is selected based on the best combined discrimination and calibration performance. Since no expert opinion is available, the choice is driven entirely by the statistical results.

The six models under evaluation are:

- **Model 1** – Full linear model
- **Model 2** – Full model with splines
- **Model 3** – Backward-selected linear model
- **Model 4** – Backward-selected model with splines
- **Model 5** – LASSO-selected linear model
- **Model 6** – LASSO-selected model with splines

Internal Validation

Discrimination

The `model$stats` output reports the **C statistic (AUC)** and **Dxy** (where $Dxy = 2 \times (C - 0.5)$), which together describe how well the model ranks predicted risks.

The AUC is the probability that, when taking a random pair of observations, the observation with $Y = 1$ (a person dying within 30 days of an AMI) has a higher predicted probability than the other. It therefore gives the probability that the model correctly ranks the risk of such pairs. From the literature, an AUC close to 1 indicates good discrimination between sensitivity and specificity.

All six models achieve $AUC > 0.80$; the full spline model (`model12`) reaches the highest value of **0.845**.

```
stats1 <- model1$stats
stats2 <- model2$stats
stats3 <- model3$stats
stats4 <- model4$stats
stats5 <- model5$stats
stats6 <- model6$stats
stats_list <- list(stats1, stats2, stats3, stats4, stats5, stats6)

discrimination_table <- data.frame(
  Model = c("Full linear", "Full spline", "Backward linear",
            "Backward spline", "LASSO linear", "LASSO spline"),
  AUC = sapply(stats_list, function(s) {round(s["C"], 3)}),
  Dxy = sapply(stats_list, function(s) {round(s["Dxy"], 3)}),
  Brier = sapply(stats_list, function(s) {round(s["Brier"], 3)}),
  Likelihood = sapply(stats_list, function(s) {round(s["Model L.R."], 3)}),
  DF = sapply(stats_list, function(s) {round(s["d.f."], 3)})
)

knitr::kable(discrimination_table,
  caption = "Internal discrimination") %>%
  kable_styling(full_width = FALSE, bootstrap_options = c("bordered"))
```

Table 6: Internal discrimination

Model	AUC	Dxy	Brier	Likelihood	DF
Full linear	0.844	0.687	0.050	300.257	17
Full spline	0.845	0.691	0.049	310.783	19
Backward linear	0.839	0.678	0.050	286.399	6
Backward spline	0.840	0.680	0.050	293.648	7
LASSO linear	0.840	0.681	0.050	296.078	8
LASSO spline	0.840	0.680	0.050	293.648	7

Shrinkage (Overfitting)

The **van Houwelingen–Le Cessie heuristic shrinkage estimate** ($\hat{c}$) quantifies overfitting. It estimates the likelihood that the model will reliably predict new observations — in other words, how much worse the model will perform on a new dataset compared to the training data. Values closer to 1 indicate less overfitting.

```

gammahat = function(s) {
  round((s["Model L.R."] - s["d.f."]) / s["Model L.R."], 3)
}

shrinkage_table <- data.frame(
  Model = c("1 - Full linear", "2 - Full spline", "3 - Backward linear",
            "4 - Backward spline", "5 - LASSO linear", "6 - LASSO spline"),

  Shrinkage = sapply(stats_list, gammahat),

  Overfitting_pct = sapply(stats_list, function(s) {
    gammaht <- gammahat(s)
    paste0(round((1 - gammaht) * 100, 1), "%")
  })
)

knitr::kable(
  shrinkage_table,
  col.names = c("Model", "Shrinkage", "Estimated overfitting"),
  caption = "Van Houwelingen-Le Cessie heuristic shrinkage estimates",
  row.names = FALSE
) %>%
  kableExtra::kable_styling(
    full_width = FALSE,
    bootstrap_options = "bordered"
  )

```

Table 7: Van Houwelingen–Le Cessie heuristic shrinkage estimates

Model	Shrinkage	Estimated overfitting
1 - Full linear	0.943	5.7%
2 - Full spline	0.939	6.1%
3 - Backward linear	0.979	2.1%
4 - Backward spline	0.976	2.4%
5 - LASSO linear	0.973	2.7%
6 - LASSO spline	0.976	2.4%

Bootstrap Validation (Discrimination and Slope Shrinkage)

The bootstrap approach is used to assess the performance of each predictive model by estimating its uncertainty and improving predictive performance estimates. The `validate()` function with $B = 200$ bootstrap resamples returns optimism-corrected Dxy and calibration slope for all six models.

The corrected Dxy (and the corresponding corrected C-index) shows how well each model would be expected to discriminate in new data, after accounting for optimism. The slope shrinkage quantifies how much the calibration slope departs from the ideal value of 1.

```

set.seed(123)
val1 <- validate(model1, B = 200)
val2 <- validate(model2, B = 200)
val3 <- validate(model3, B = 200)

```

```

val4 <- validate(model4, B = 200)
val5 <- validate(model5, B = 200)
val6 <- validate(model6, B = 200)

extract_val <- function(val) {
  c(
    Apparent_Dxy = round(val["Dxy", "index.orig"], 3),
    Corrected_Dxy = round(val["Dxy", "index.corrected"], 3),
    Slope_Shrinkage = round(val["Slope", "index.orig"] - val["Slope", "index.corrected"], 3)
  )
}

val_table <- data.frame(
  Model = c("1 - Full linear", "2 - Full spline", "3 - Backward linear",
            "4 - Backward spline", "5 - LASSO linear", "6 - LASSO spline"),
  do.call(rbind, lapply(list(val1, val2, val3, val4, val5, val6), extract_val))
)

kable(val_table,
      col.names = c("Model", "Apparent Dxy", "Corrected Dxy", "Slope Shrinkage"),
      caption = "Internal Validation: Bootstrap selection") %>%
  kable_styling(full_width = FALSE, bootstrap_options = c("bordered"))

```

Table 8: Internal Validation: Bootstrap selection

Model	Apparent Dxy	Corrected Dxy	Slope Shrinkage
1 - Full linear	0.687	0.659	0.065
2 - Full spline	0.691	0.659	0.073
3 - Backward linear	0.678	0.670	0.022
4 - Backward spline	0.680	0.670	0.022
5 - LASSO linear	0.681	0.671	0.027
6 - LASSO spline	0.680	0.671	0.022

Model 2 (Full spline) has the highest Dxy, but does not have the best (smallest) slope shrinkage. The backward-spline and LASSO-spline models generally show the lowest slope shrinkage, suggesting better generalisability.

Calibration (Bootstrap)

The `calibrate()` function estimates the mean absolute error (MAE) between estimated probabilities and observed event rates. Lower MAE indicates better calibration.

```

cal1 <- calibrate(model11, B = 200)
cal2 <- calibrate(model12, B = 200)
cal3 <- calibrate(model13, B = 200)
cal4 <- calibrate(model14, B = 200)
cal5 <- calibrate(model15, B = 200)
cal6 <- calibrate(model16, B = 200)

```

Models 4 and 6 (backward-spline and LASSO-spline) achieve the lowest MAE, indicating the best internal calibration.

```

extract_cal_mae <- function(cal) {
  p      <- cal[, "predy"]
  p.cal <- cal[, "calibrated.corrected"]
  predicted <- attr(cal, "predicted")

  s      <- !is.na(p + p.cal)
  err <- predicted - approx(p[s], p.cal[s], xout = predicted, ties = mean)$y

  round(mean(abs(err), na.rm = TRUE), 3)
}

cal_table <- data.frame(
  Model = c("1 - Full linear", "2 - Full spline", "3 - Backward linear",
            "4 - Backward spline", "5 - LASSO linear", "6 - LASSO spline"),
  MAE = c(
    extract_cal_mae(cal1), extract_cal_mae(cal2),
    extract_cal_mae(cal3), extract_cal_mae(cal4),
    extract_cal_mae(cal5), extract_cal_mae(cal6)
  )
)

knitr::kable(cal_table,
  col.names = c("Model", "Mean Absolute Error"),
  caption = "Internal calibration: MAE") %>%
  kable_styling(full_width = FALSE, bootstrap_options = "bordered")

```

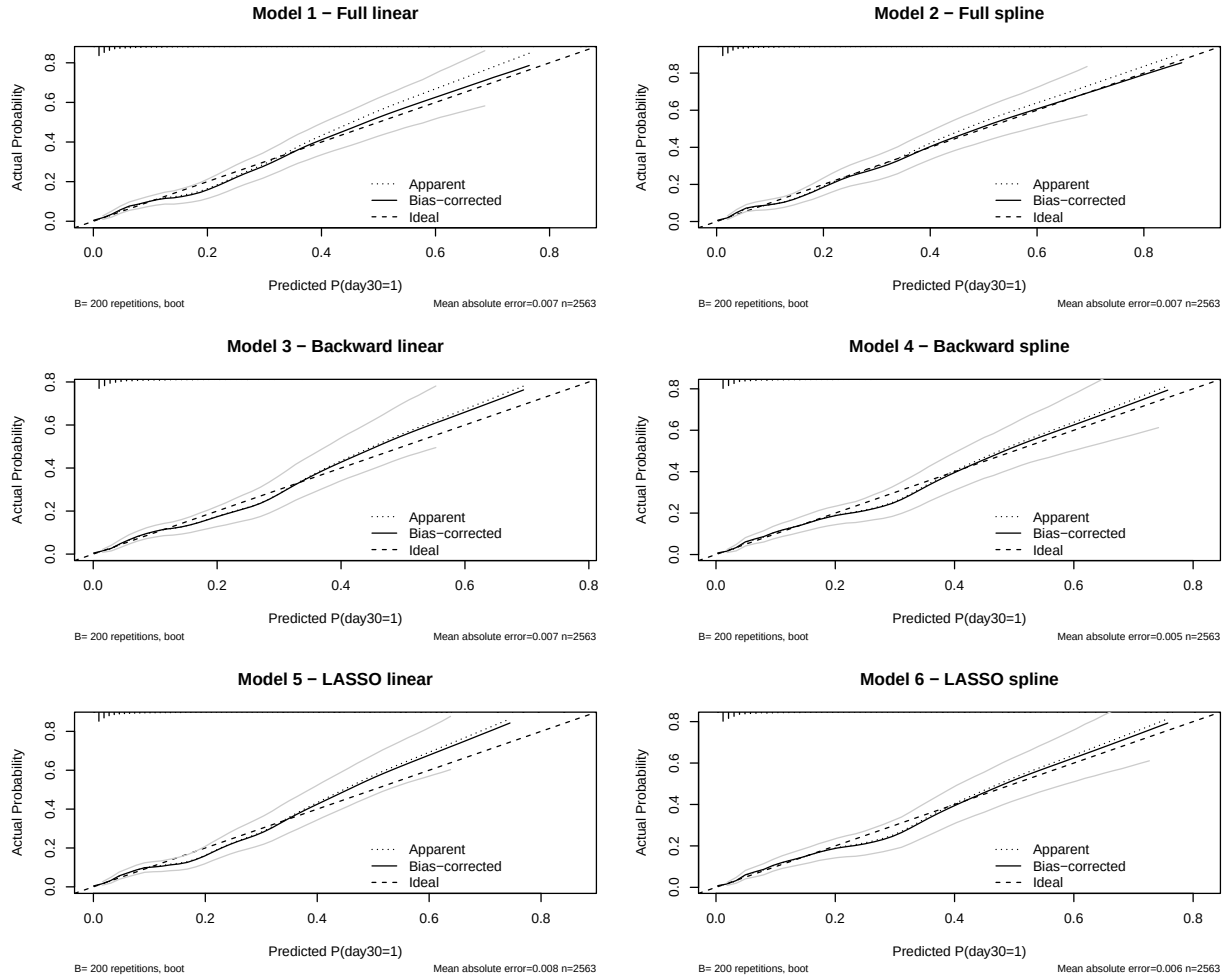
Table 9: Internal calibration: MAE

Model	Mean Absolute Error
1 - Full linear	0.007
2 - Full spline	0.007
3 - Backward linear	0.007
4 - Backward spline	0.005
5 - LASSO linear	0.008
6 - LASSO spline	0.006

```

par(mfrow = c(3, 2))
plot(cal1); title(main = "Model 1 - Full linear")
plot(cal2); title(main = "Model 2 - Full spline")
plot(cal3); title(main = "Model 3 - Backward linear")
plot(cal4); title(main = "Model 4 - Backward spline")
plot(cal5); title(main = "Model 5 - LASSO linear")
plot(cal6); title(main = "Model 6 - LASSO spline")

```



AIC values across models further support the preference for spline and reduced models:

```
cal_table <- data.frame(
  Model = c("1 - Full linear", "2 - Full spline", "3 - Backward linear",
            "4 - Backward spline", "5 - LASSO linear", "6 - LASSO spline"),
  AIC = round(c(
    AIC(model11), AIC(model12), AIC(model13),
    AIC(model14), AIC(model15), AIC(model16)
  ), 2)
)

knitr::kable(cal_table,
  col.names = c("Model", "AIC"), caption = "Internal calibration: AIC") %>%
  kable_styling(full_width = FALSE, bootstrap_options = c("bordered"))
```

Table 10: Internal calibration: AIC

Model	AIC
1 - Full linear	976.07
2 - Full spline	969.55
3 - Backward linear	967.93

4 - Backward spline	962.68
5 - LASSO linear	962.25
6 - LASSO spline	962.68

Taken together, the internal validation results indicate that the backward-selection spline and LASSO-spline models offer the best combination of discrimination, low overfitting, and good calibration.

External Validation

External validation assesses prediction performance on the held-out **test set**, providing an unbiased estimate of how each model performs on new data. Predicted probabilities are first extracted from each model and then used to compute the AUC and to evaluate calibration.

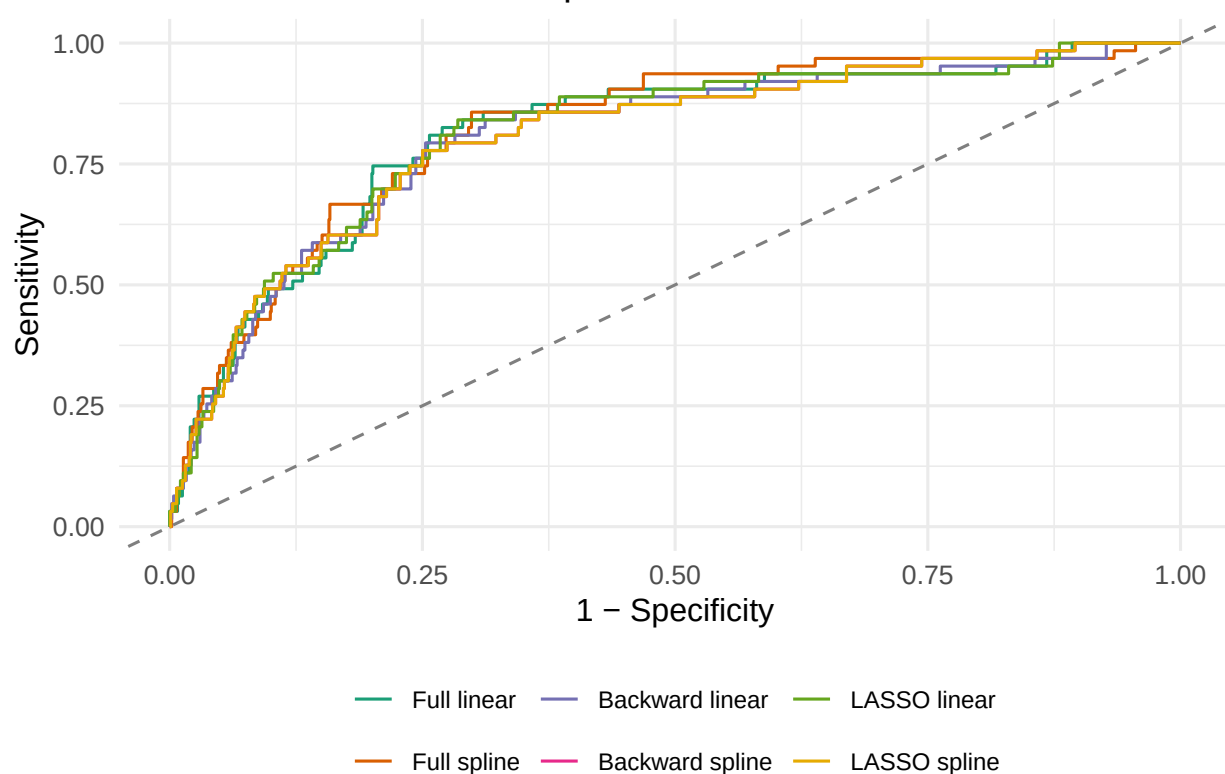
Discrimination on the Test Set

```
test1 <- predict(model1, newdata = test, type = "fitted")
test2 <- predict(model2, newdata = test, type = "fitted")
test3 <- predict(model3, newdata = test, type = "fitted")
test4 <- predict(model4, newdata = test, type = "fitted")
test5 <- predict(model5, newdata = test, type = "fitted")
test6 <- predict(model6, newdata = test, type = "fitted")

roc_list <- list(
  "Full linear"      = roc(test$day30, test1, quiet = TRUE),
  "Full spline"     = roc(test$day30, test2, quiet = TRUE),
  "Backward linear" = roc(test$day30, test3, quiet = TRUE),
  "Backward spline" = roc(test$day30, test4, quiet = TRUE),
  "LASSO linear"    = roc(test$day30, test5, quiet = TRUE),
  "LASSO spline"    = roc(test$day30, test6, quiet = TRUE)
)

ggroc(roc_list, legacy.axes = TRUE) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey50") +
  scale_colour_brewer(palette = "Dark2") +
  labs(
    title = "ROC Curves - Model Comparison on Test Dataset",
    x = "1 - Specificity",
    y = "Sensitivity",
    colour = NULL
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
        legend.text = element_text(size = 9))
```

ROC Curves – Model Comparison on Test Dataset



```
auc_table <- data.frame(
  Model = names(roc_list),
  AUC   = round(sapply(roc_list, pROC::auc), 3)
)

kable(auc_table, caption = "AUC", row.names = FALSE) %>% kable_styling(full_width = FALSE, bootstrap_opt = FALSE)
```

Table 11: AUC

Model	AUC
Full linear	0.813
Full spline	0.822
Backward linear	0.804
Backward spline	0.805
LASSO linear	0.811
LASSO spline	0.805

The AUC shows that the model with best performance is the full model with spline (`model2`), though all the others also show good values above 0.80. Full models show marginally higher discrimination, which may be related to greater model flexibility.

However, differences in AUC across models are small, suggesting broadly comparable discriminative performance.

Calibration

Calibration was assessed both graphically using smoothed calibration curves and formally using the Hosmer–Lemeshow test.

```
par(mfrow = c(3, 2))
cali1 <- val.prob(test1, test$day30, g = 10)
title(main = "Model 1 - Full linear")

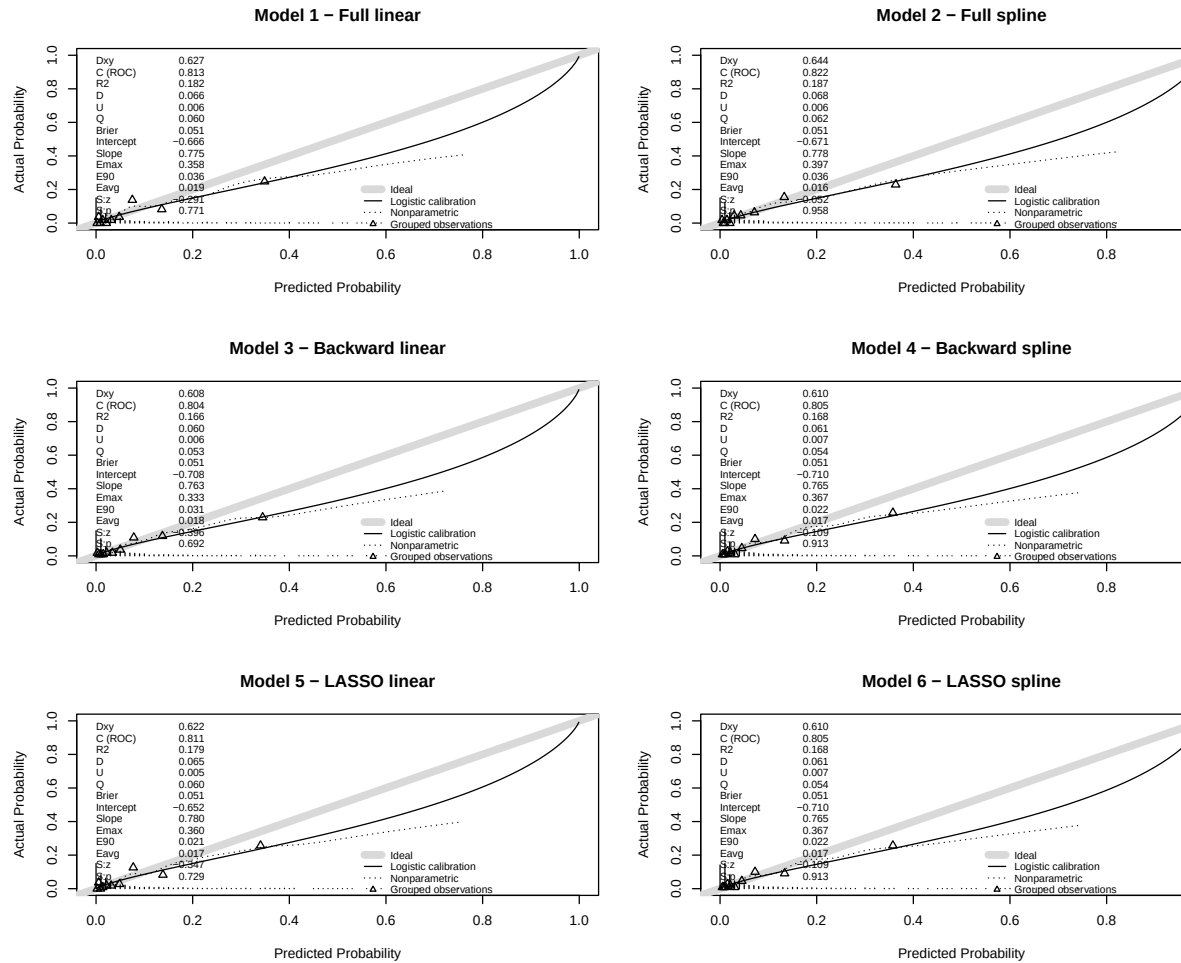
cali2 <- val.prob(test2, test$day30, g = 10)
title(main = "Model 2 - Full spline")

cali3 <- val.prob(test3, test$day30, g = 10)
title(main = "Model 3 - Backward linear")

cali4 <- val.prob(test4, test$day30, g = 10)
title(main = "Model 4 - Backward spline")

cali5 <- val.prob(test5, test$day30, g = 10)
title(main = "Model 5 - LASSO linear")

cali6 <- val.prob(test6, test$day30, g = 10)
title(main = "Model 6 - LASSO spline")
```



Visual Calibration

Hosmer–Lemeshow Test The **Hosmer–Lemeshow (HL)** test formally assesses whether the predicted probabilities match the observed event rates across deciles of risk.

- **p-value** > 0.05 indicates that the model is well calibrated (no significant difference between predicted and observed)
- **p-value** ≤ 0.05 signals systematic miscalibration.

```
pred_list <- list(
  "1 - Full linear"      = test1,
  "2 - Full spline"     = test2,
  "3 - Backward linear" = test3,
  "4 - Backward spline" = test4,
  "5 - LASSO linear"    = test5,
  "6 - LASSO spline"    = test6
)

hoslem_results <- data.frame(
  Model = names(pred_list),
  ChiSq = NA_real_,
  p_value = NA_real_,

```

```

  Calibration = NA_character_
)

for (i in seq_along(pred_list)) {
  hl <- hoslem.test(test$day30, pred_list[[i]])

  hoslem_results$ChiSq[i] <- round(unname(hl$statistic), 3)
  hoslem_results$p_value[i] <- round(hl$p.value, 4)

  hoslem_results$Calibration[i] <- ifelse(
    hl$p.value > 0.05,
    "Well-calibrated",
    "Poor calibration"
  )
}

knitr::kable(
  hoslem_results,
  caption = "Hosmer-Lemeshow calibration test across models",
  row.names = FALSE
) %>%
  kableExtra::kable_styling(
    full_width = FALSE,
    bootstrap_options = "bordered"
  )

```

Table 12: Hosmer–Lemeshow calibration test across models

Model	ChiSq	p_value	Calibration
1 - Full linear	36.112	0.0000	Poor calibration
2 - Full spline	19.590	0.0120	Poor calibration
3 - Backward linear	19.709	0.0115	Poor calibration
4 - Backward spline	11.060	0.1983	Well-calibrated
5 - LASSO linear	28.608	0.0004	Poor calibration
6 - LASSO spline	11.060	0.1983	Well-calibrated

Based on the Hosmer–Lemeshow test, **Model 4 (Backward spline)** and **Model 6 (LASSO spline)** are well calibrated ($p > 0.05$). The remaining models show statistically significant miscalibration on the test set.

Conclusion

Based on both internal and external validation, the **Backward-selected model with splines (Model 4)** is selected as the final predictive model. Its key advantages are:

- Good test-set AUC (0.805), comparable to more complex models.
- Lowest slope shrinkage among spline models, indicating strong generalisability.
- Mean absolute calibration error of 0.005 (joint lowest internally).
- **Well-calibrated** on the test set per the Hosmer–Lemeshow test ($p > 0.05$), unlike the full models.
- Parsimonious: uses only 6 predictors with spline terms for age, reducing overfitting risk compared to the 17-predictor full models.

Table 5: Table 1. Baseline characteristics by 30-day vital status

Characteristic	After 30 days		
	Overall N = 3,661	Alive N = 3,430	Dead N = 231
Age (years), Median (Q1, Q3)	61 (52, 70)	60 (51, 69)	73 (65, 81)
Killip class, n (%)			
1	3,166 (86%)	3,022 (88%)	144 (62%)
2	433 (12%)	376 (11%)	57 (25%)
3	43 (1.2%)	24 (0.7%)	19 (8%)
4	19 (0.5%)	8 (0.2%)	11 (5%)
Shock (Killip 3/4), n (%)	62 (1.7%)	32 (0.9%)	30 (13%)
Anterior infarct location, n (%)	1,347 (37%)	1,215 (35%)	132 (57%)
Tachycardia (pulse > 80), n (%)	1,161 (32%)	1,043 (30%)	118 (51%)
ST elevation (n leads), Median (Q1, Q3)	4.00 (3.00, 5.00)	4.00 (3.00, 5.00)	5.00 (3.00, 6.00)
ST elevation > 4 leads, n (%)	1,349 (37%)	1,227 (36%)	122 (53%)
Smoking status, n (%)			
Never	1,451 (40%)	1,393 (41%)	58 (25%)
Ex	1,185 (32%)	1,099 (32%)	86 (37%)
Current	1,025 (28%)	938 (27%)	87 (38%)
Hypotension (SBP < 100), n (%)	319 (8.7%)	274 (8.0%)	45 (19%)
Previous MI, n (%)	617 (17%)	548 (16%)	69 (30%)
Sex, n (%)			
Male	2,721 (74%)	2,581 (75%)	140 (60%)
Female	940 (26%)	849 (25%)	91 (39%)
Body Mass Index (kg/m ²), Median (Q1, Q3)	26.8 (24.2, 29.9)	26.9 (24.3, 30.0)	25.3 (22.7, 28.0)
Previous angina, n (%)	1,282 (35%)	1,176 (34%)	106 (46%)
Time to pain relief > 1h, n (%)	2,158 (59%)	1,998 (58%)	160 (69%)
Diabetes, n (%)	481 (13%)	440 (13%)	41 (18%)
Family history of MI, n (%)	1,672 (46%)	1,580 (46%)	92 (40%)
Hypertension history, n (%)	1,451 (40%)	1,348 (39%)	103 (44%)
Hypercholesterolaemia, n (%)	1,455 (40%)	1,372 (40%)	83 (36%)

¹Standardized Mean Difference

Abbreviation: CI = Confidence Interval

Characteristic	N	OR	95% CI	p-value
Age (years)	3,661	1.10	1.08, 1.11	<0.001
Killip class	3,661			
1		—	—	
2		3.18	2.28, 4.38	<0.001
3		16.6	8.80, 31.0	<0.001
4		28.9	11.5, 75.6	<0.001
Shock (Killip 3/4)	3,661			
No		—	—	
Yes		15.8	9.41, 26.6	<0.001
Anterior infarct location	3,661			
No		—	—	
Yes		2.43	1.86, 3.19	<0.001
Tachycardia (pulse > 80)	3,661			
No		—	—	
Yes		2.39	1.83, 3.13	<0.001
ST elevation (n leads)	3,661	1.20	1.12, 1.29	<0.001
ST elevation > 4 leads	3,661			
No		—	—	
Yes		2.01	1.54, 2.63	<0.001
Smoking status	3,661			
Never		—	—	
Ex		1.88	1.34, 2.66	<0.001
Current		2.23	1.59, 3.15	<0.001
Hypotension (SBP < 100)	3,661			
No		—	—	
Yes		2.79	1.95, 3.91	<0.001
Previous MI	3,661			
No		—	—	
Yes		2.24	1.66, 3.00	<0.001
Sex	3,661			
Male		—	—	
Female		1.98	1.50, 2.60	<0.001
Body Mass Index (kg/m ²)	3,661	0.92	0.90, 0.95	<0.001
Previous angina	3,661			
No		—	—	
Yes		1.63	1.24, 2.12	<0.001
Time to pain relief > 1h	3,661			
No		—	—	
Yes		1.62	1.22, 2.16	0.001
Diabetes	3,661			
No		—	—	
Yes		1.47	1.02, 2.06	0.033
Family history of MI	3,661			
No		—	—	
Yes		0.77	0.59, 1.02	0.066
Hypertension history	3,661			
No	50	—	—	
Yes		1.24	0.95, 1.62	0.11
Hypercholesterolaemia	3,661			
No		—	—	